

Lecture 2

The Internet

Subjects of today:

- The Internet
- The Network edge
- The Network core
- Performance
- The Protocol layers of the Internet
- Lab exercise

The Internet: a “nuts and bolts” view



Billions of connected computing *devices*:

- *hosts* = end systems
- running *network apps* at Internet's “edge”



Packet switches: forward packets (chunks of data)

- *routers, switches*

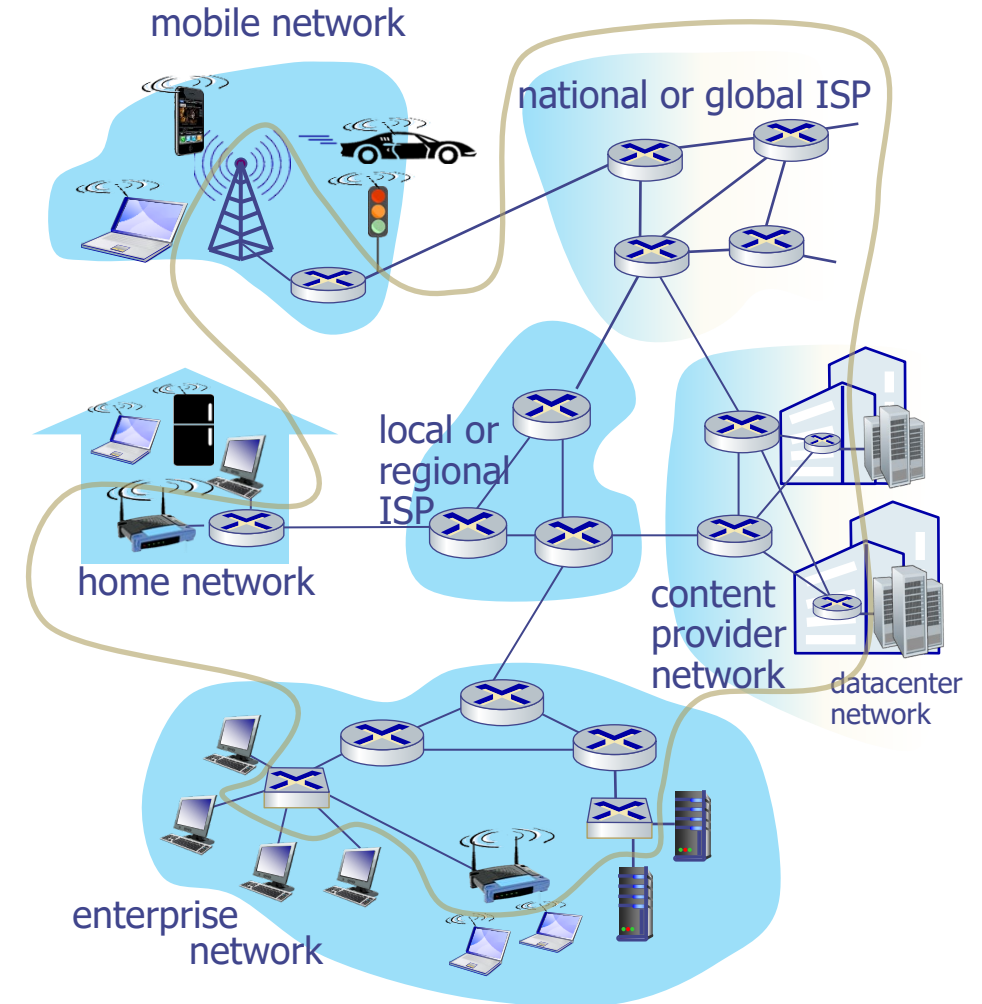
Communication links

- fiber, copper, radio, satellite
- transmission rate: *bandwidth*



Networks

- collection of devices, routers, links: managed by an organization



“Fun” Internet-connected devices



Amazon Echo



Internet refrigerator



IP picture frame



Pacemaker & Monitor



Tweet-a-watt:
monitor energy use



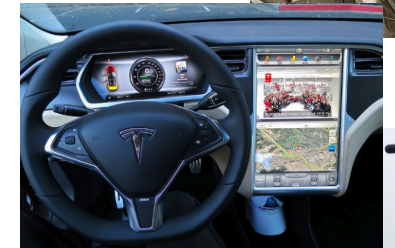
bikes



Security Camera



Web-enabled toaster +
weather forecaster



cars



AR devices



scooters



Internet phones



Gaming devices



sensorized
bed
mattress



Fitbit

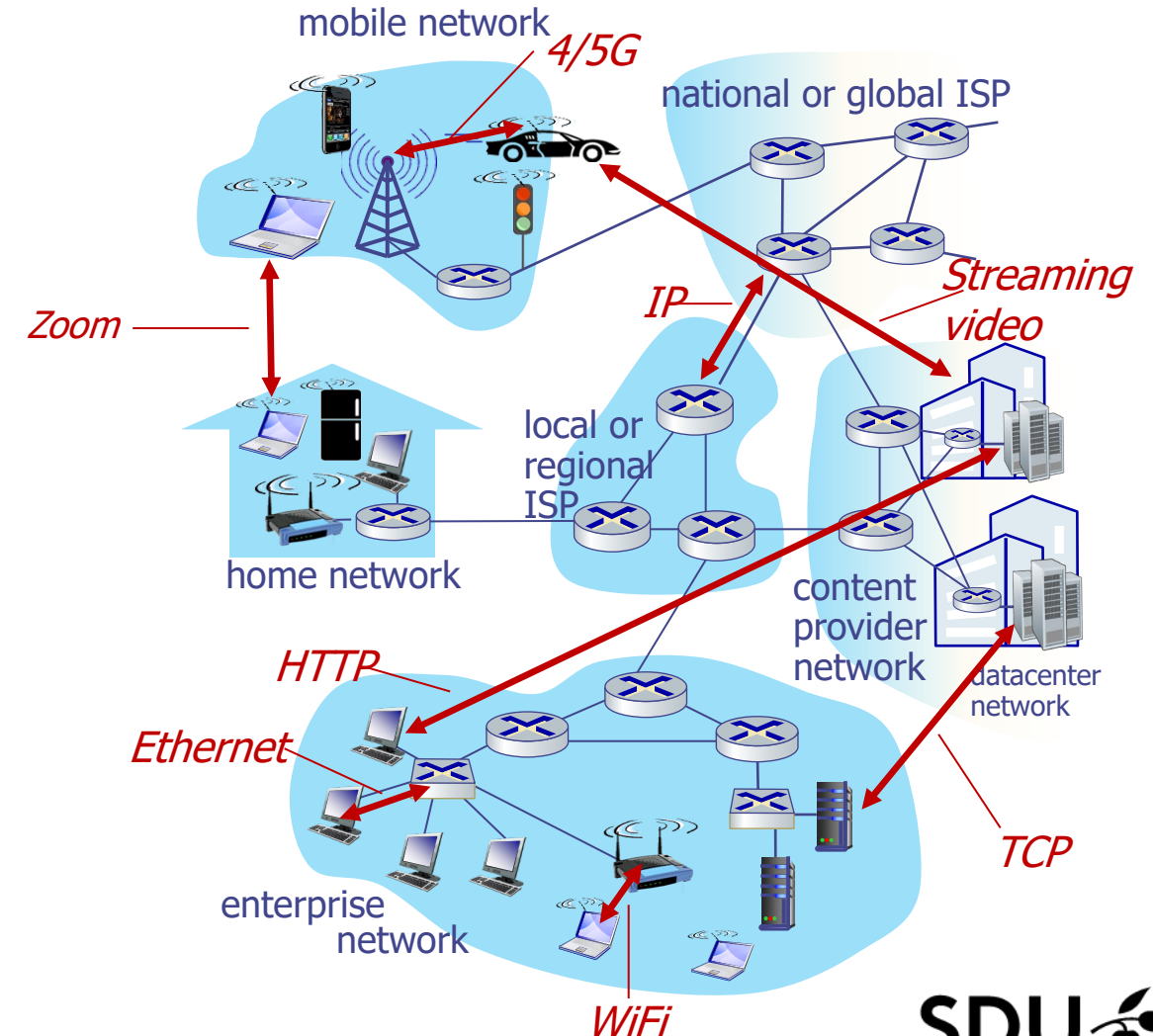


diapers

Others?

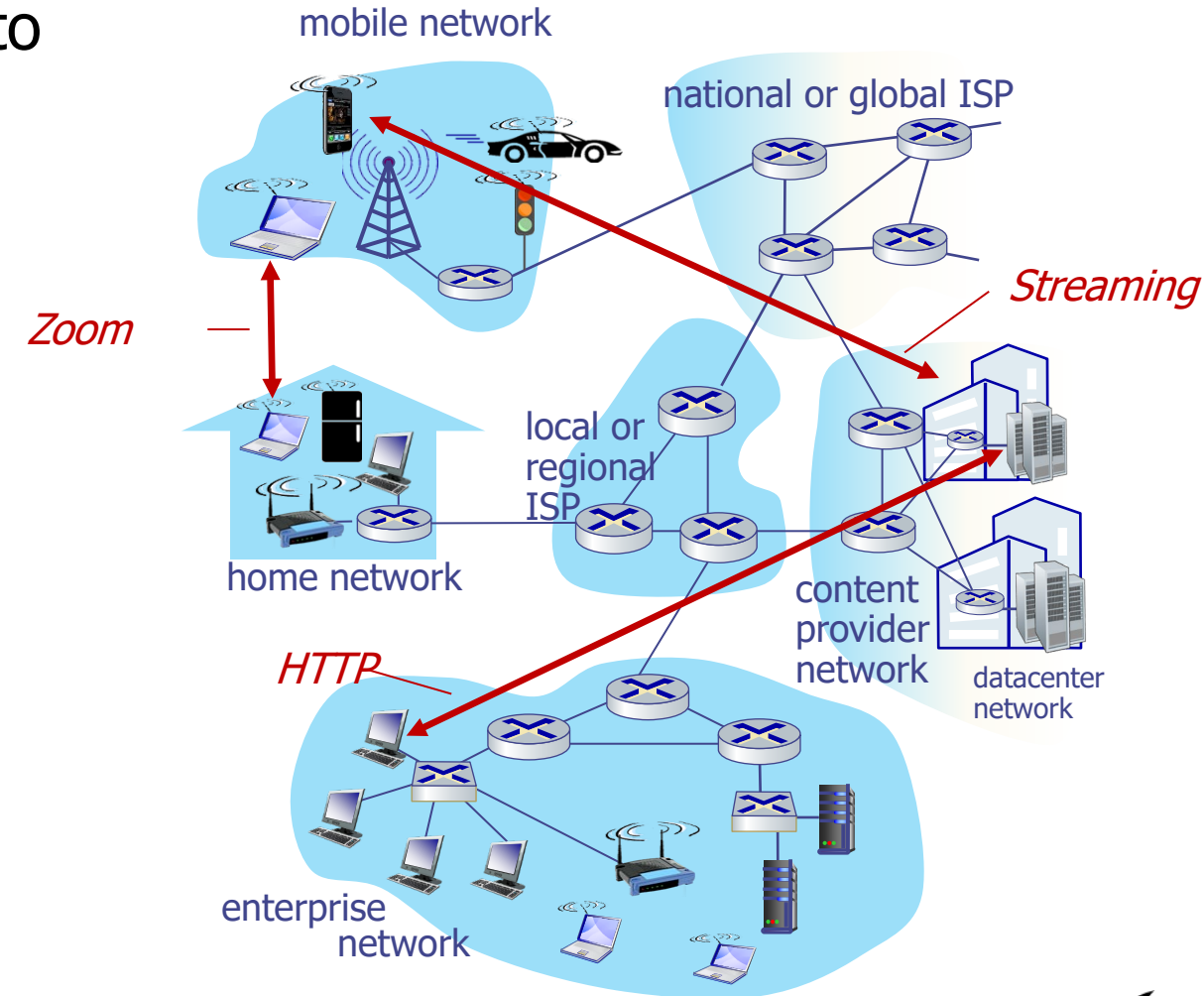
The Internet: a “nuts and bolts” view

- *Internet: "network of networks"*
 - Interconnected ISPs
- *protocols are everywhere*
 - control sending, receiving of messages
 - e.g., HTTP (Web), streaming video, Zoom, TCP, IP, WiFi, 4/5G, Ethernet
- *Internet standards*
 - RFC: Request for Comments
 - IETF: Internet Engineering Task Force



The Internet: a “services” view

- *Infrastructure* that provides services to applications
- Provides *programming interface* to distributed applications
- The internet core **transfers messages** from one end system to another end system



Protocols, a reminder

Human protocols:

- “what’s the time?”
- “I have a question”
- introductions

Rules for:

- ... specific messages sent
- ... specific actions taken
when message received,
or other events

Network protocols:

- computers (devices) rather than humans
- all communication activity in Internet
governed by protocols

*Protocols define the format, order
of messages sent and received
among network entities, and
actions taken on message
transmission, receipt*

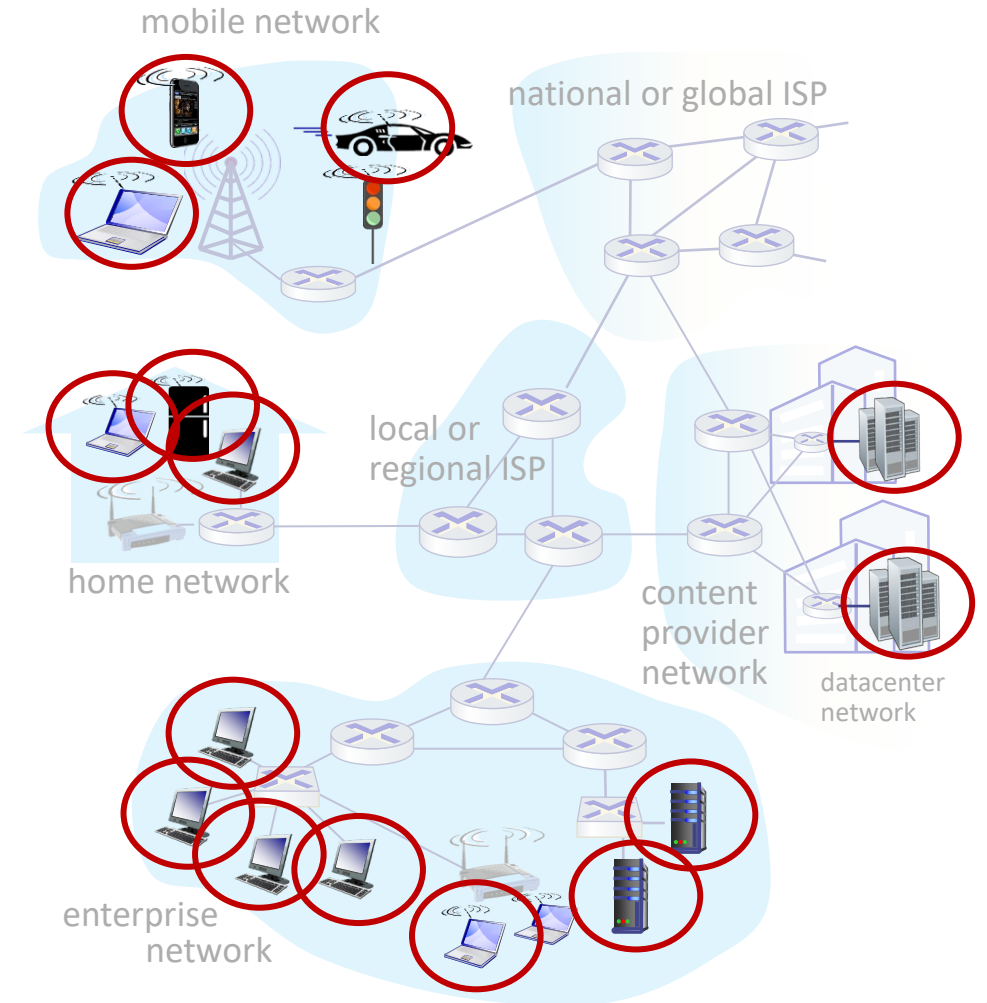
Subjects of today:

- The Internet
- **The Network edge**
- The Network core
- Performance
- The Protocol layers of the Internet

A closer look at Internet structure

Network edge:

- hosts: clients and servers
- servers often in data centers



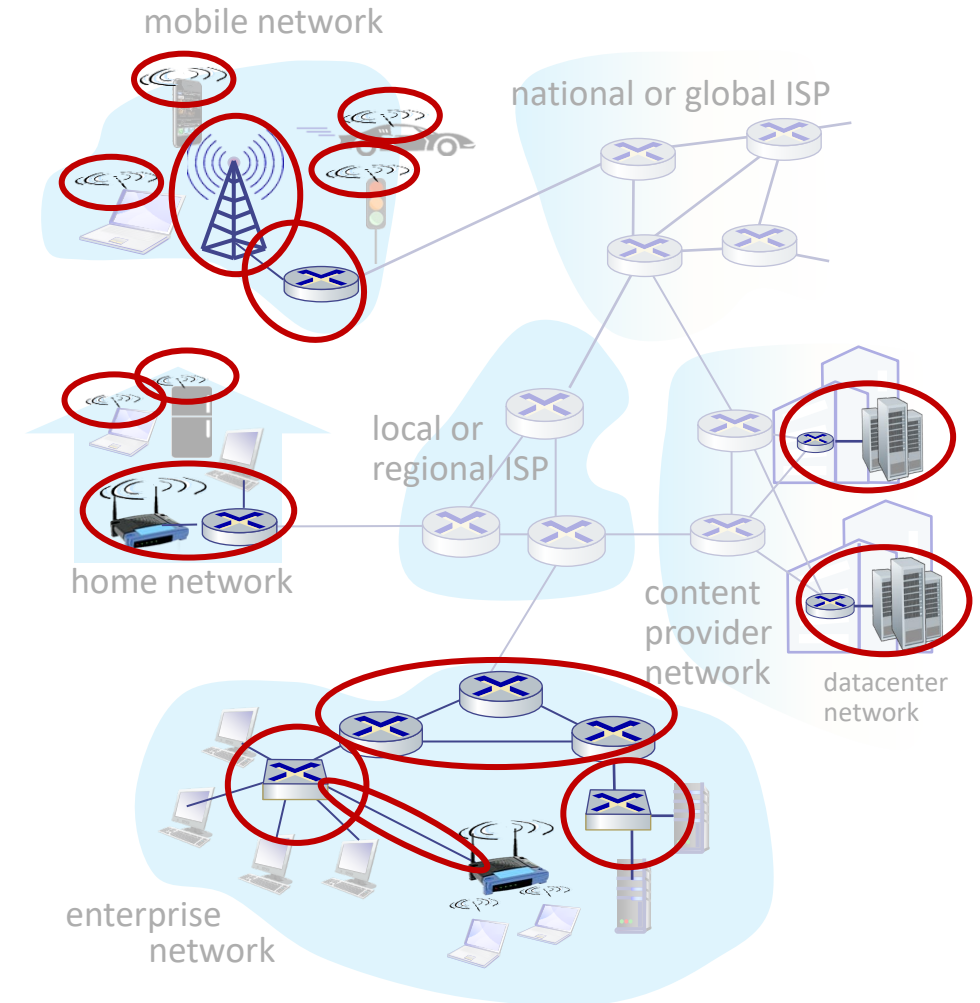
A closer look at Internet structure

Network edge:

- hosts: clients and servers
- servers often in data centers

Access networks, physical media:

- wired, wireless communication links



A closer look at Internet structure

Network edge:

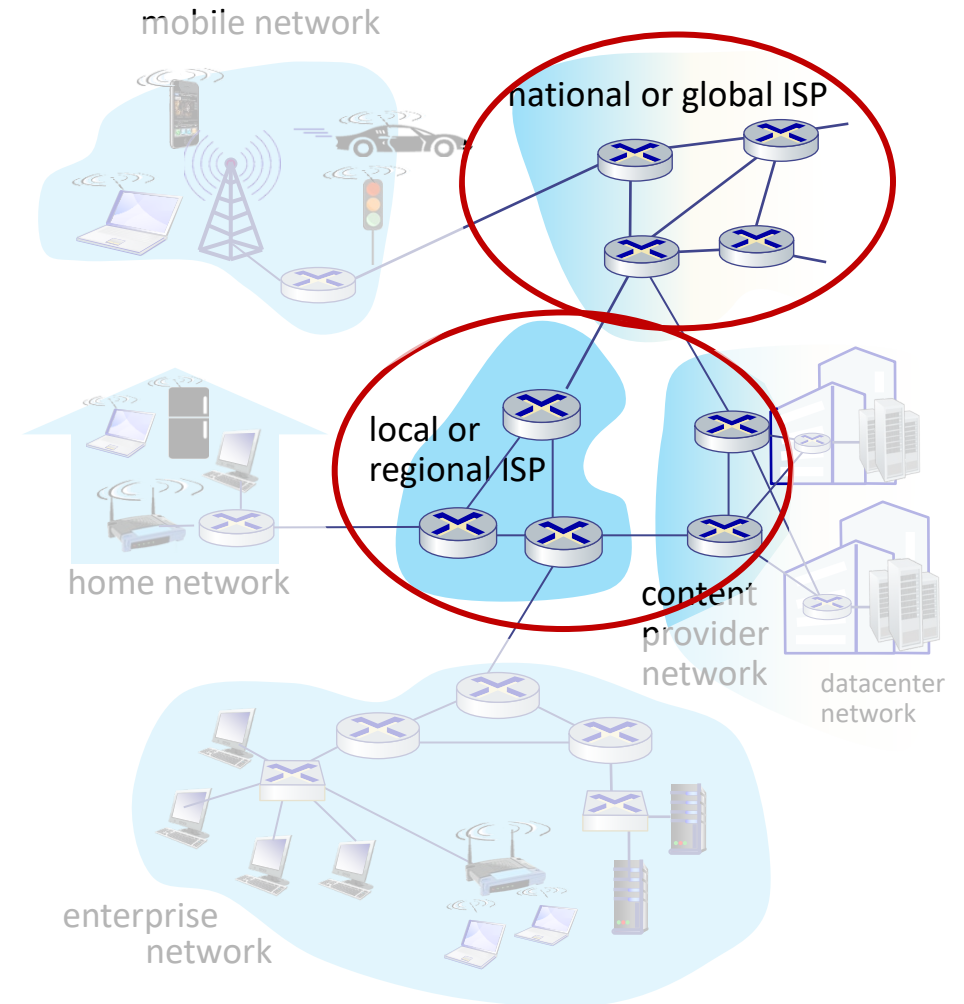
- hosts: clients and servers
- servers often in data centers

Access networks, physical media:

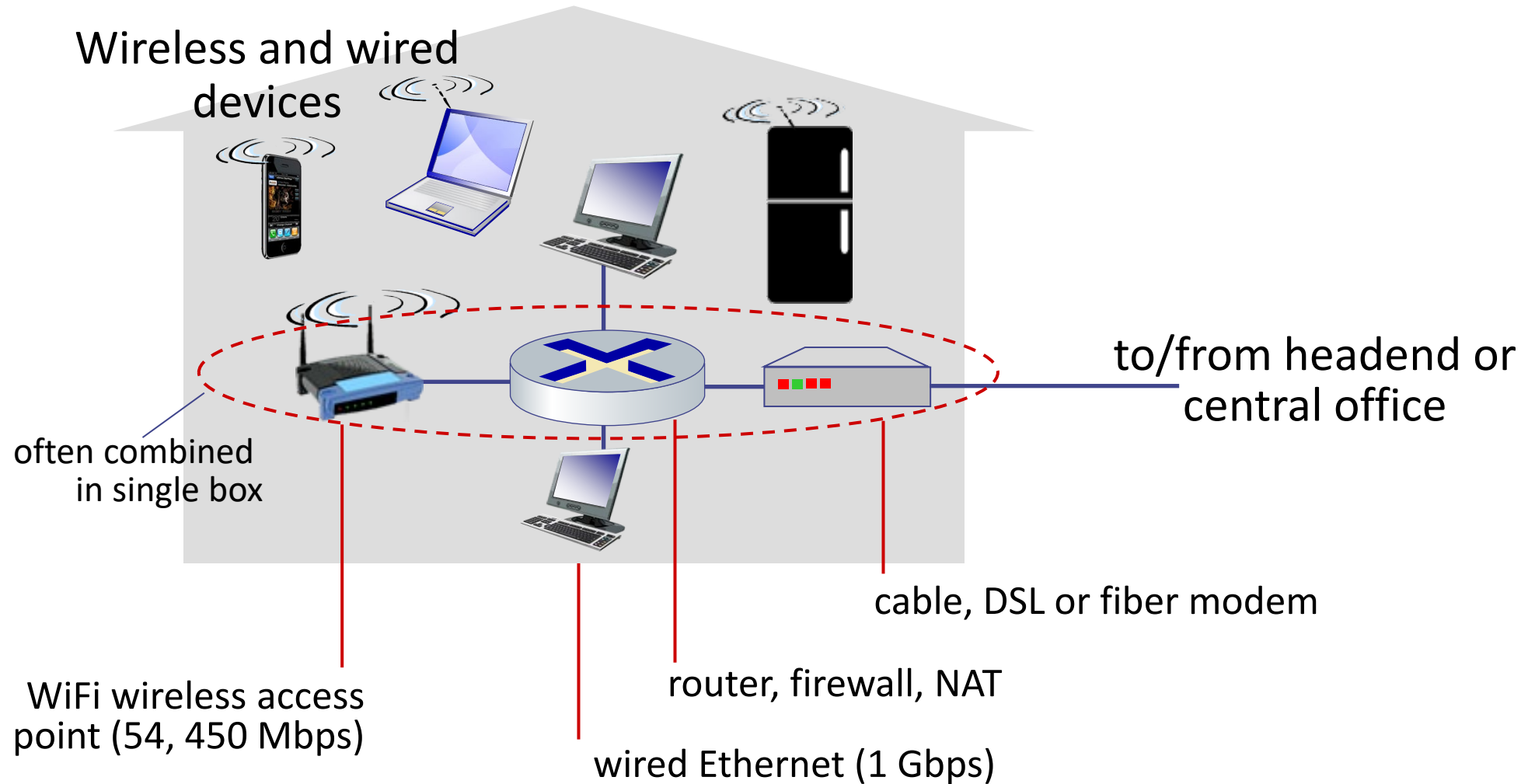
- wired, wireless communication links

Network core:

- interconnected routers
- network of networks



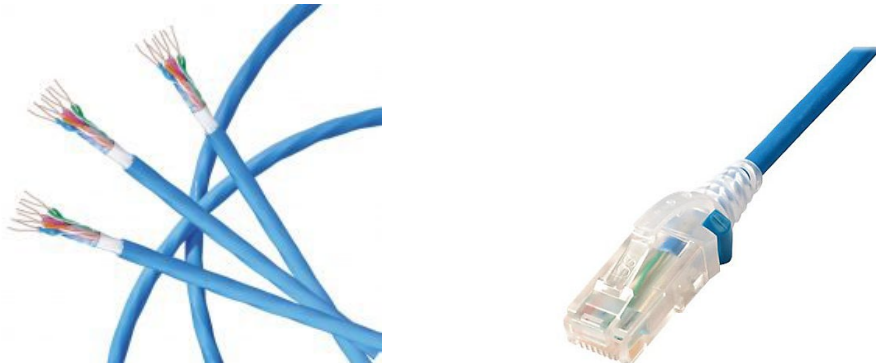
Access networks: home networks



Links: physical media

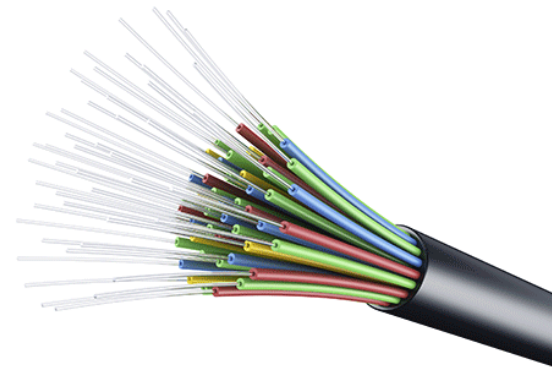
Twisted pair (TP)

- two insulated copper wires
 - Category 5: 100 Mbps, 1 Gbps Ethernet
 - Category 6: 10Gbps Ethernet



Fiber optic cable:

- glass fiber carrying light pulses, each pulse a bit
- high-speed operation:
 - high-speed point-to-point transmission (10's-100's Gbps)
- low error rate:
 - repeaters spaced far apart
 - immune to electromagnetic noise

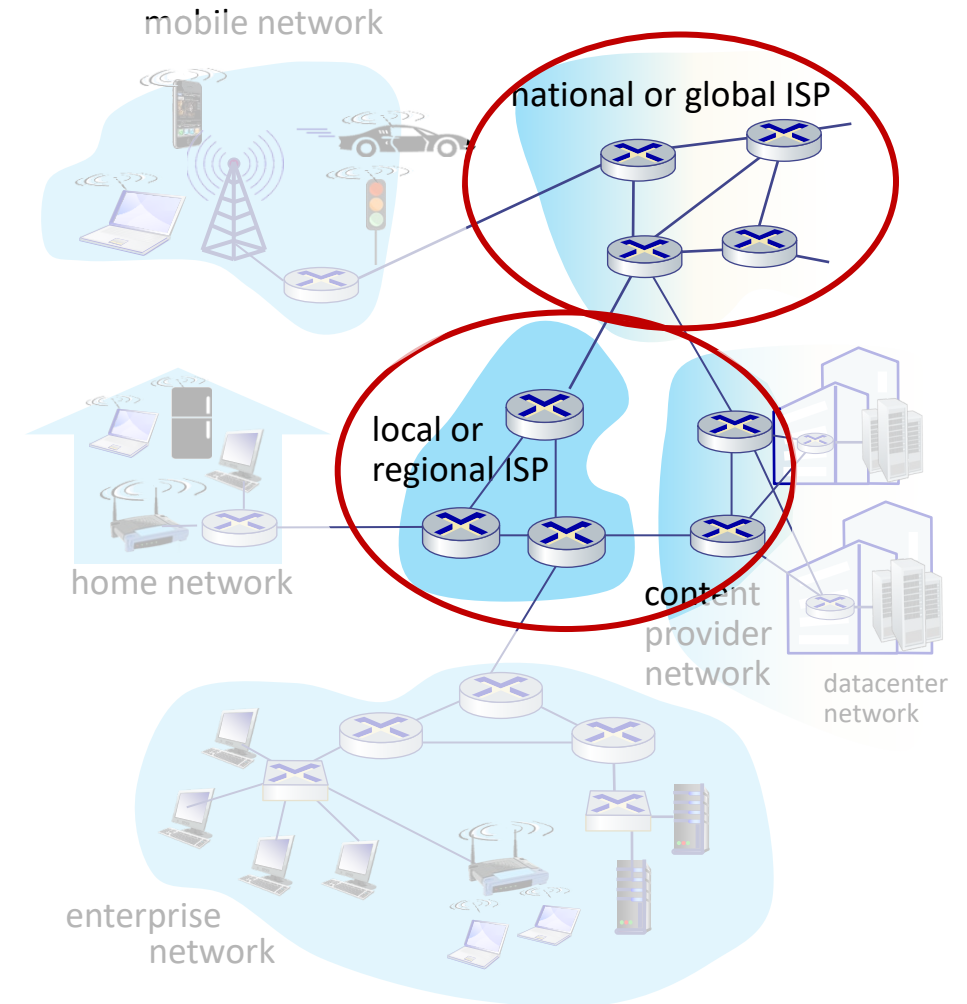


Subjects of today:

- The Internet
- The Network edge
- **The Network core**
- Performance
- The Protocol layers of the Internet
- Lab exercise

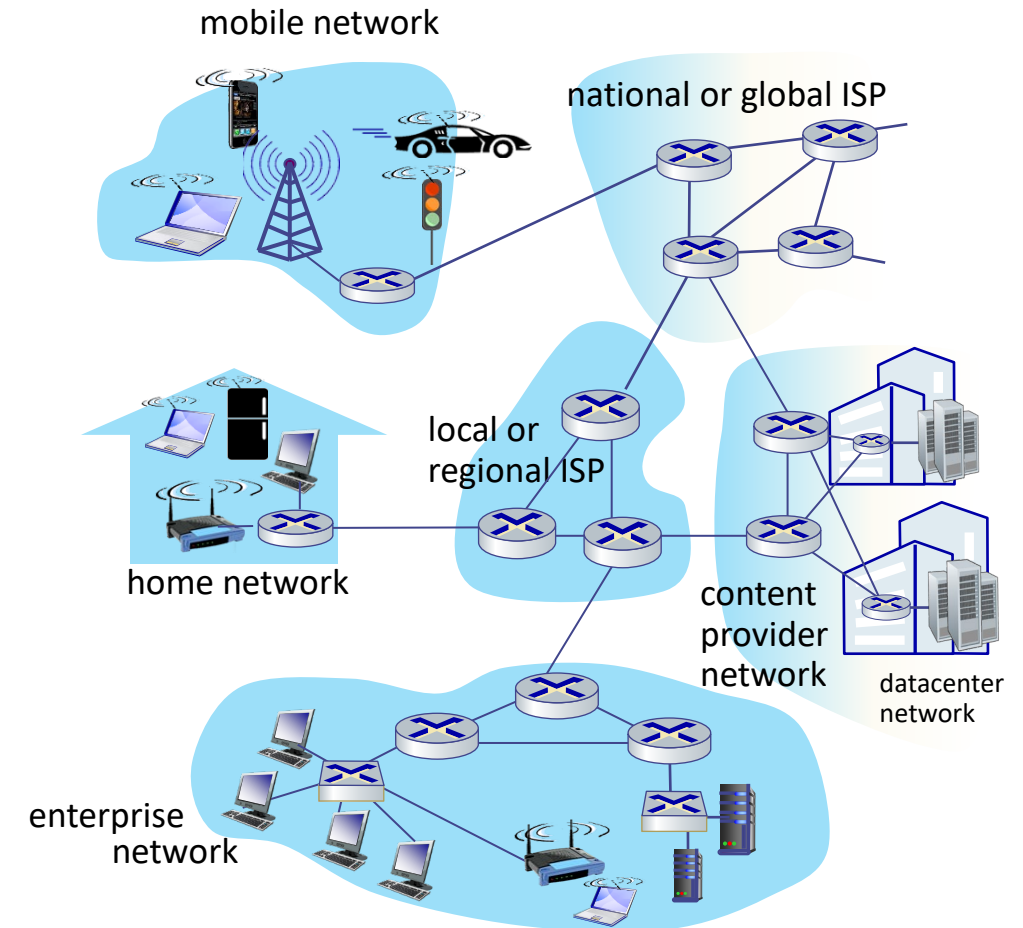
The network core

- mesh of interconnected routers
- **packet-switching**: hosts break application-layer messages into *packets*
 - network **forwards** packets from one router to the next, across links on path from **source to destination**



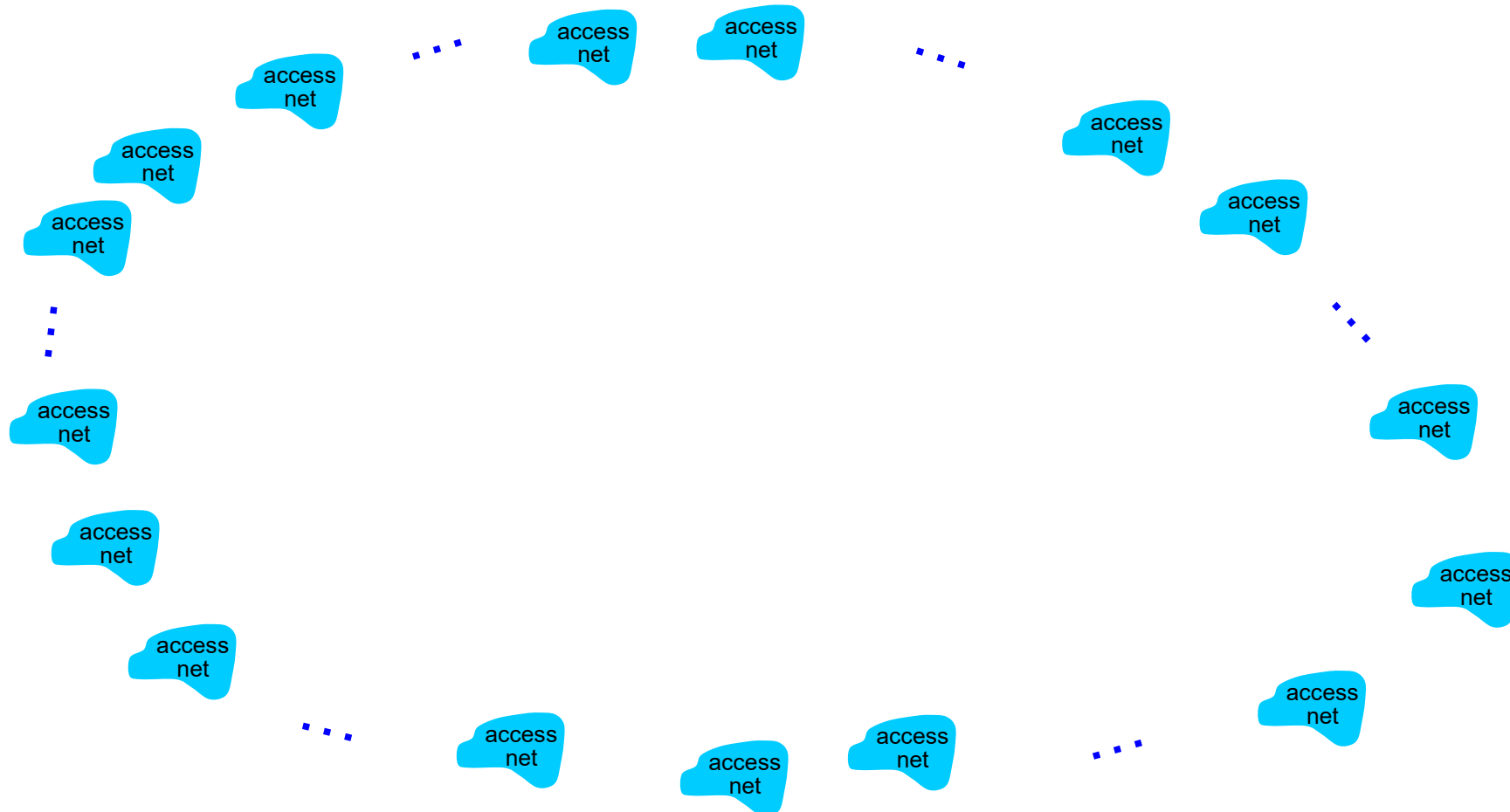
Internet structure: a “network of networks”

- hosts connect to Internet via **access** Internet Service Providers (ISPs)
- access ISPs in turn must be interconnected



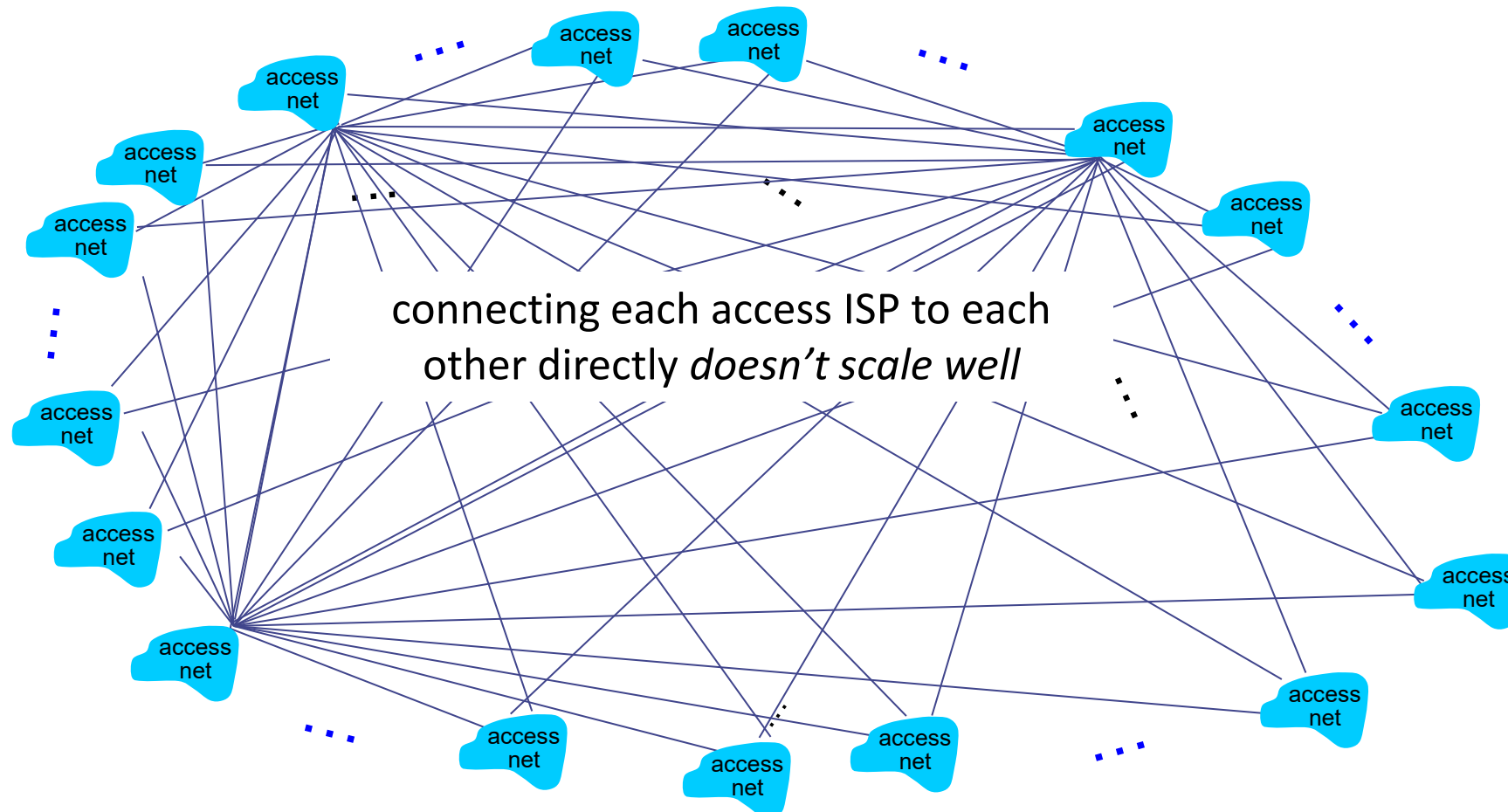
Internet structure: a “network of networks”

Given *millions* of access ISPs, how to connect them together?



Internet structure: a “network of networks”

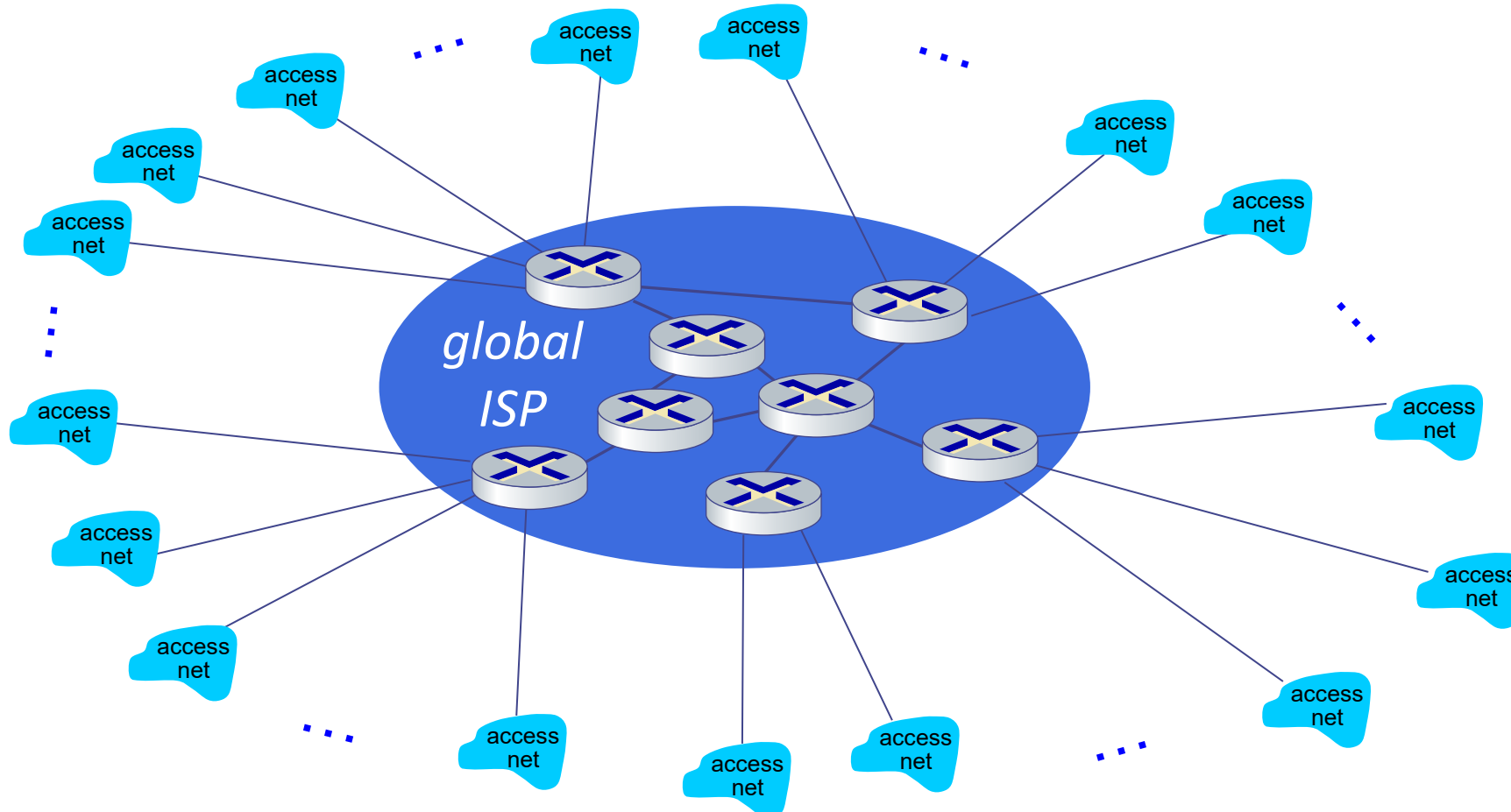
Given *millions* of access ISPs, how to connect them together?



Internet structure: a “network of networks”

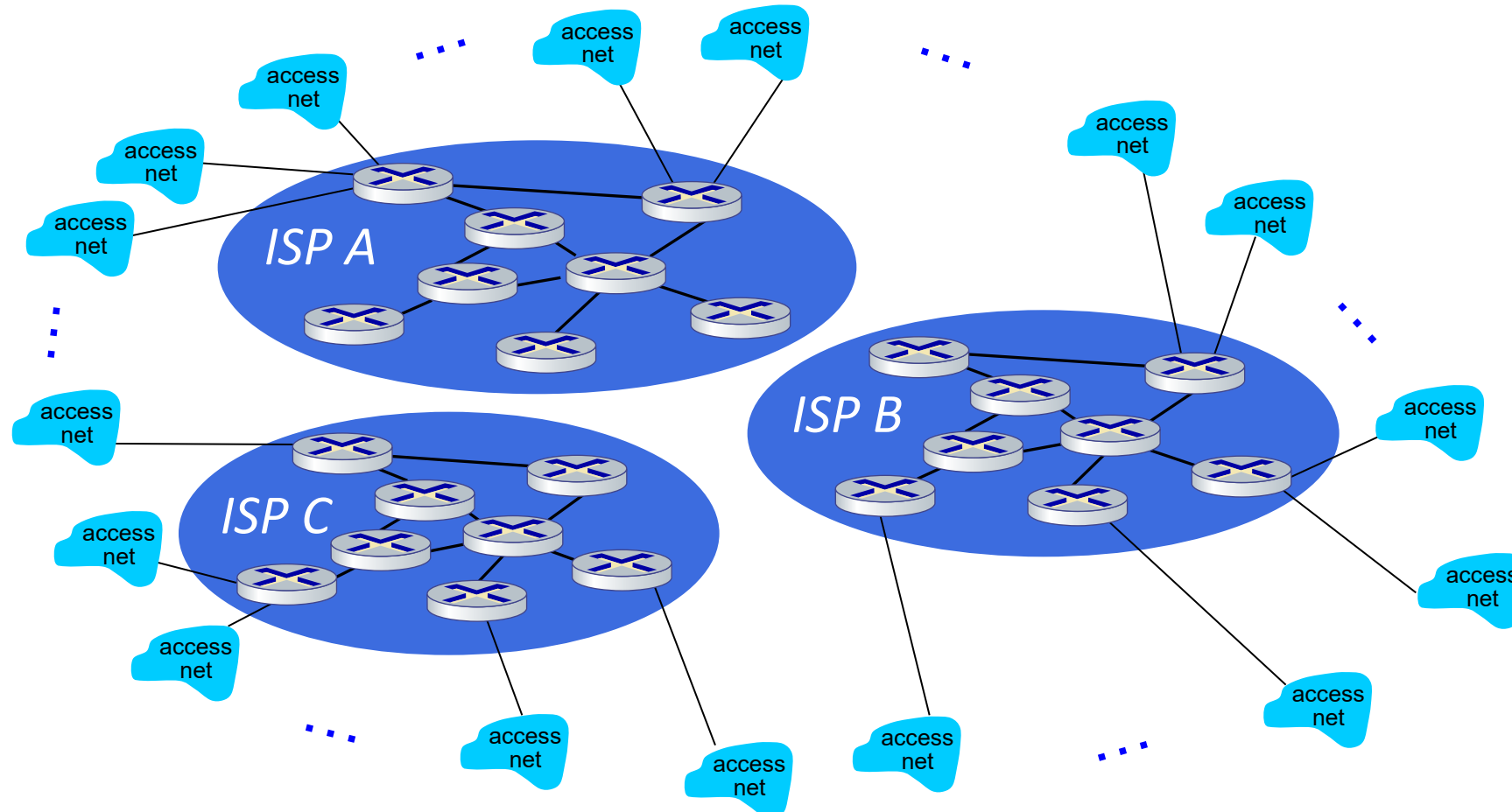
Option: connect each access ISP to one global transit ISP?

Customer and provider ISPs have economic agreement.



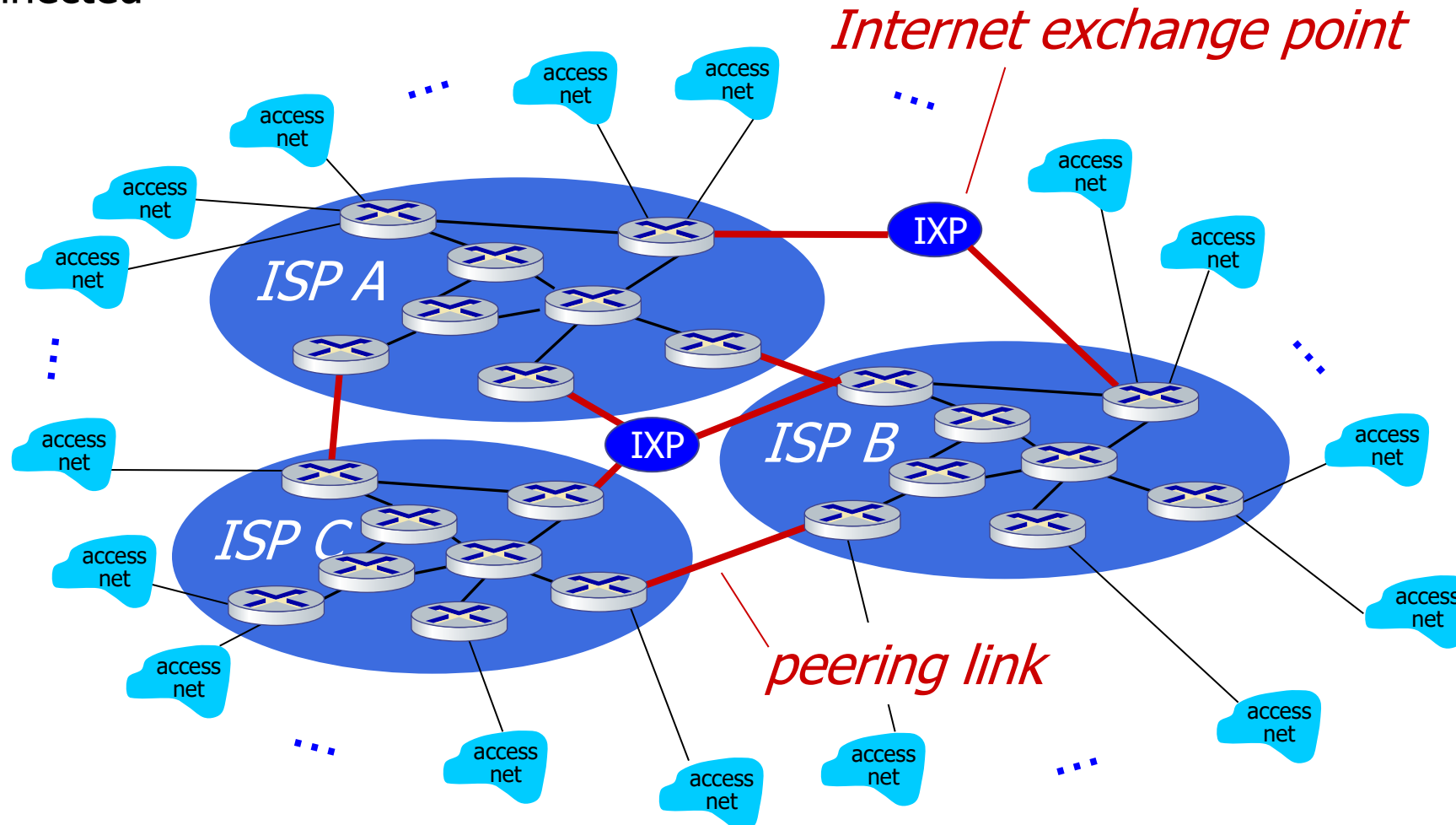
Internet structure: a “network of networks”

But if one global ISP is viable business, there will be competitors



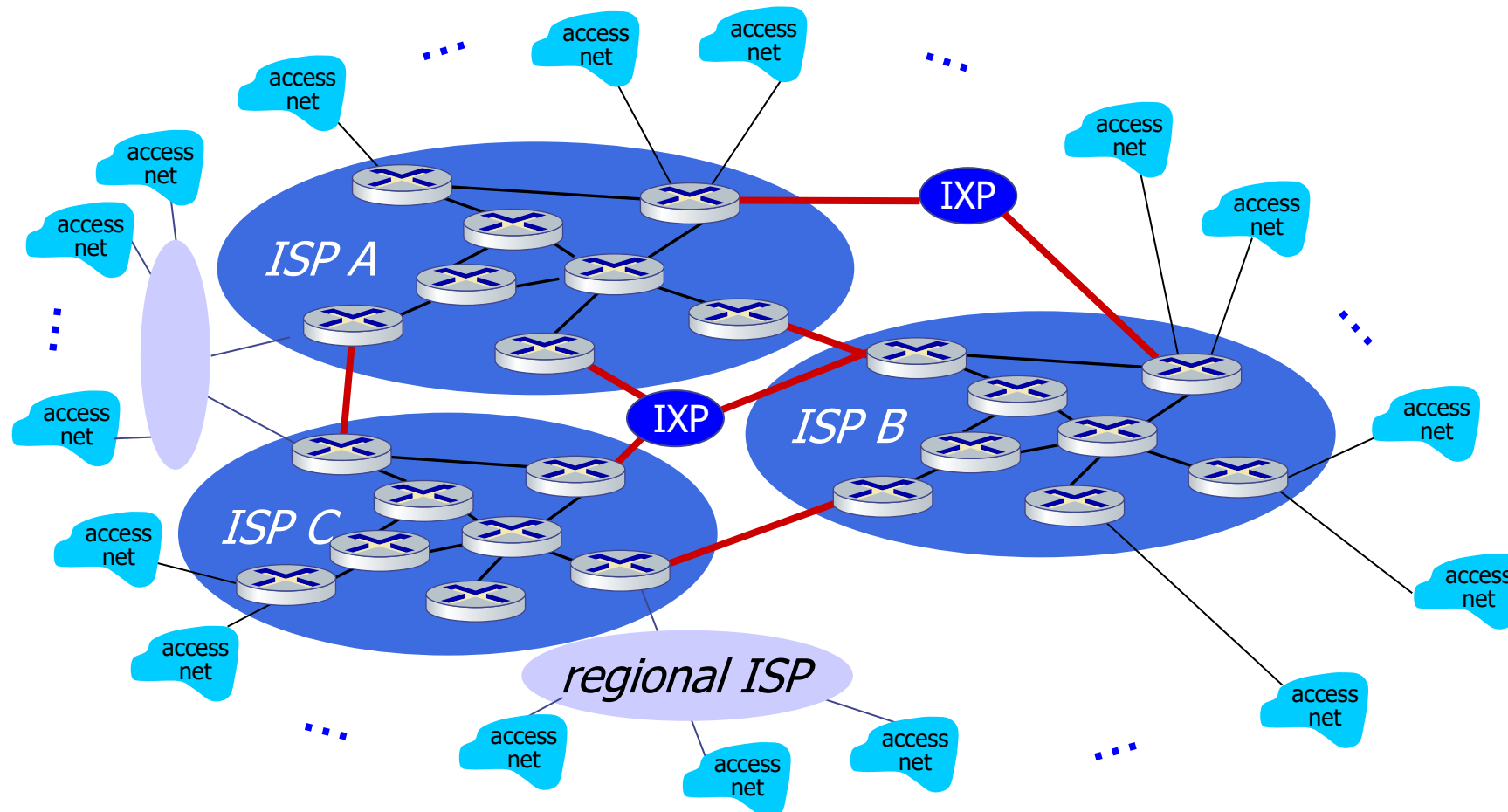
Internet structure: a “network of networks”

But if one global ISP is viable business, there will be competitors who will want to be connected



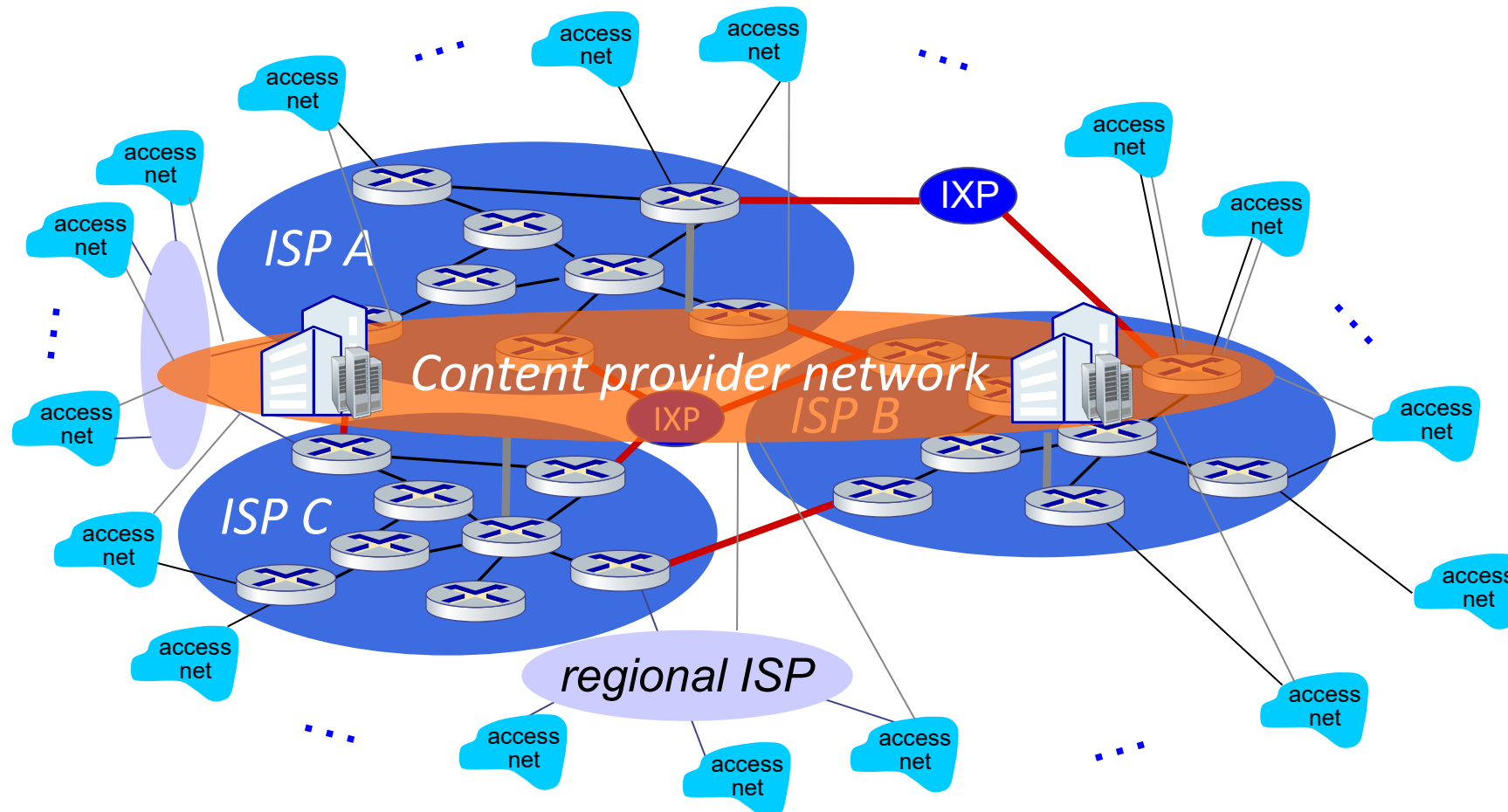
Internet structure: a “network of networks”

... and regional networks may arise to connect access nets to ISPs

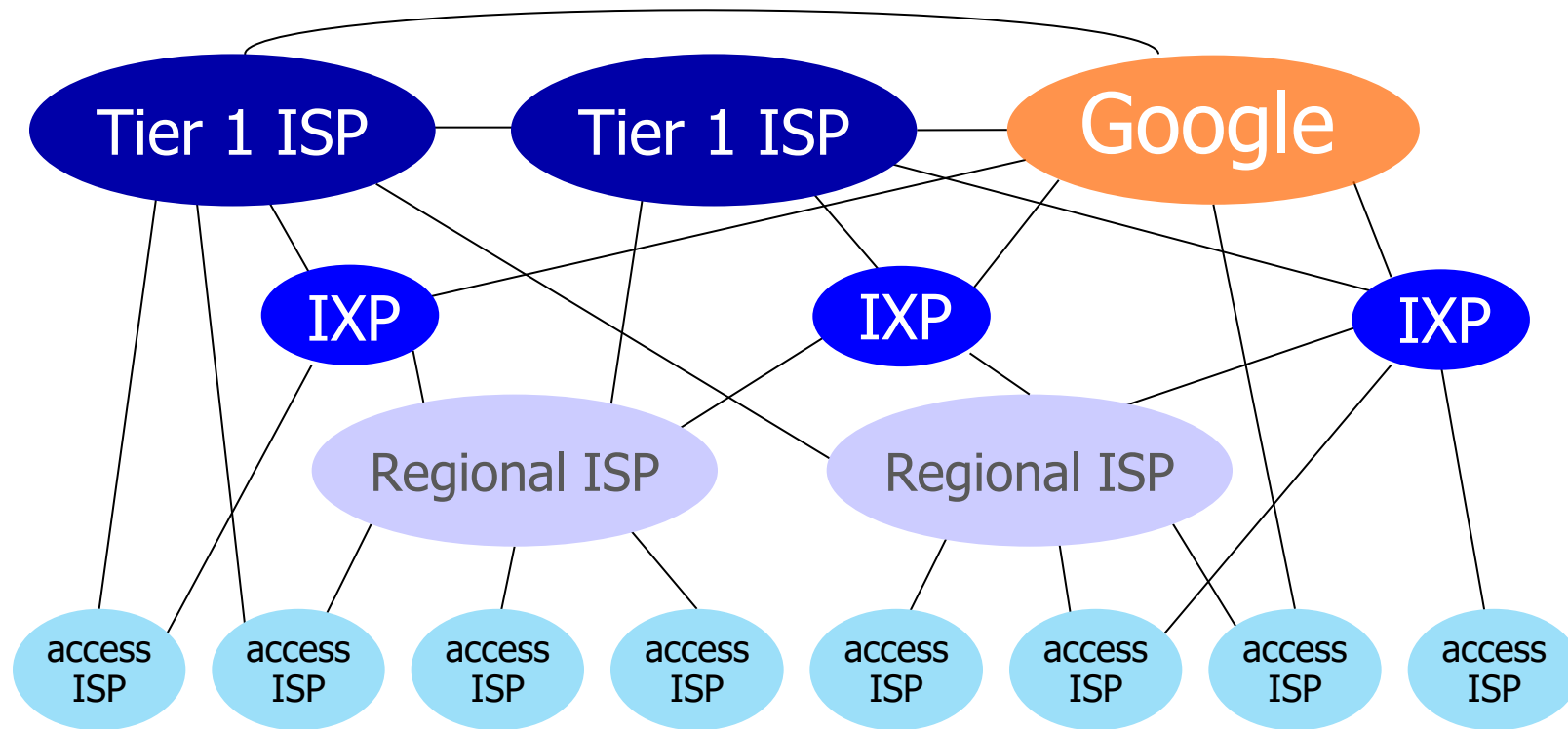


Internet structure: a “network of networks”

... and content provider networks (e.g., Google, Microsoft, Akamai) may run their own network, to bring services, content close to end users



Internet structure: a “network of networks”



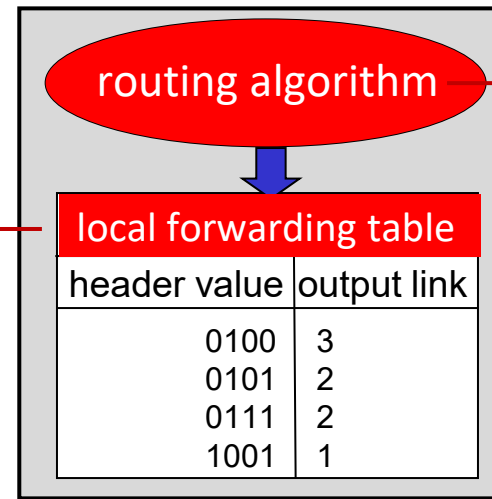
At “center”: small # of well-connected large networks

- **“tier-1” commercial ISPs** (e.g., Level 3, Sprint, AT&T, NTT), national & international coverage
- **content provider networks** (e.g., Google, Facebook): private network that connects its data centers to Internet, often bypassing tier-1, regional ISPs

Two key network-core functions

Forwarding:

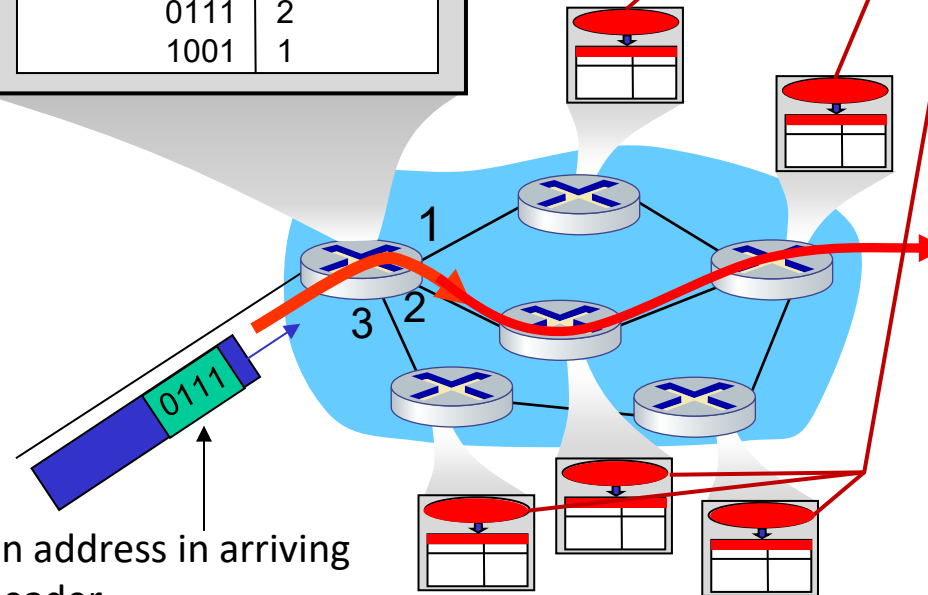
- aka "switching"
- *local* action: move arriving packets from router's input link to appropriate router output link



destination address in arriving packet's header

Routing:

- *global* action: determine source-destination paths taken by packets
- routing algorithms

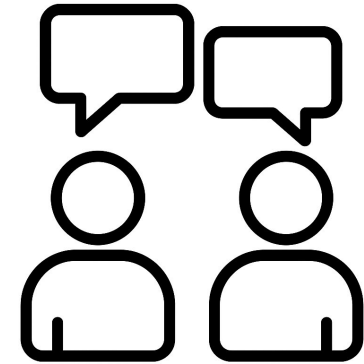






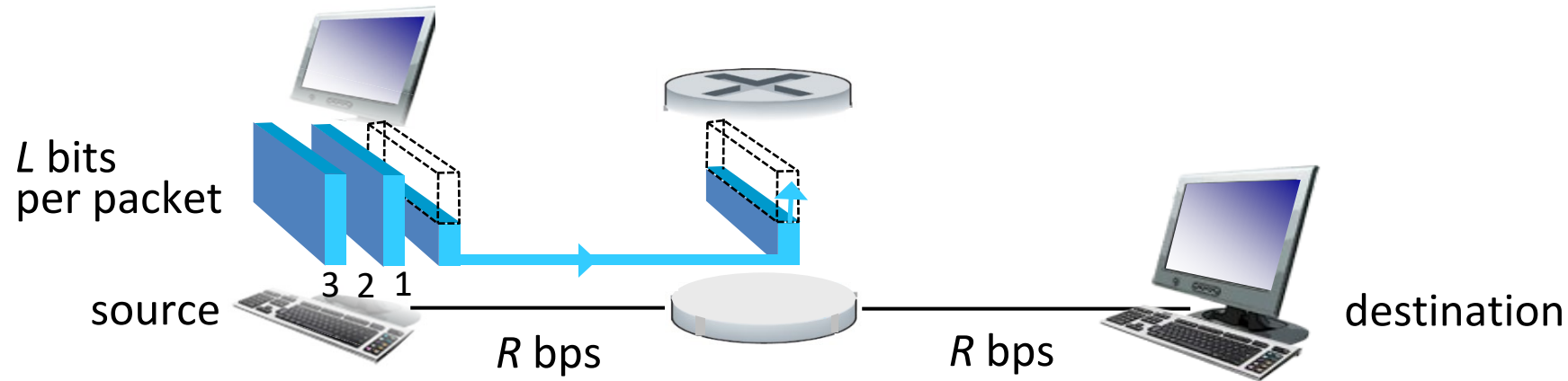
IP-adresses

- 32 bits → 4 billion possible addresses
- Addressing:
 - Binary: xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx
 - E.g.: 11000000 10101000 00000001 00001100
 - Decimal: ddd.ddd.ddd.ddd
 - E.g.: 192.168.1.12
- Interpreted left to right



Do you see
any problems
with this?

Packet-switching: store-and-forward

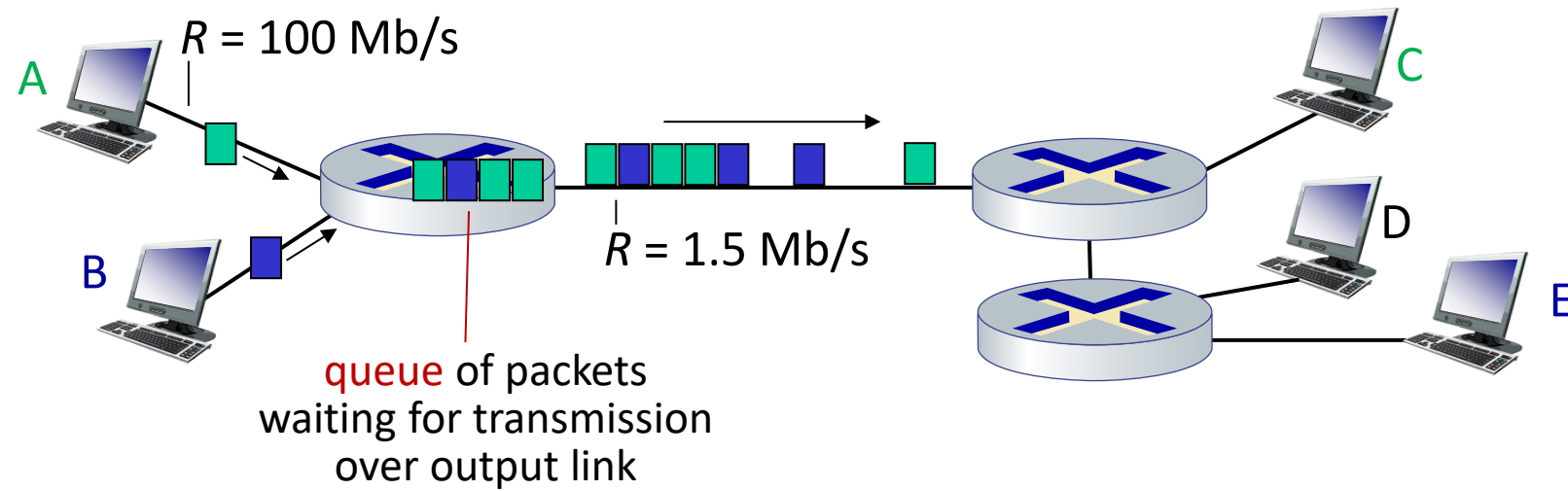


- **packet transmission delay:** takes L/R seconds to transmit (push out) L -bit packet into link at R bps
- **store and forward:** *entire* packet must arrive at router before it can be transmitted on next link

One-hop numerical example:

- $L = 10$ Kbits
- $R = 100$ Mbps
- one-hop transmission delay = 0.1 msec

Packet-switching: queueing



Packet queuing and loss: if arrival rate (in bps) to link exceeds transmission rate (bps) of link for some period of time:

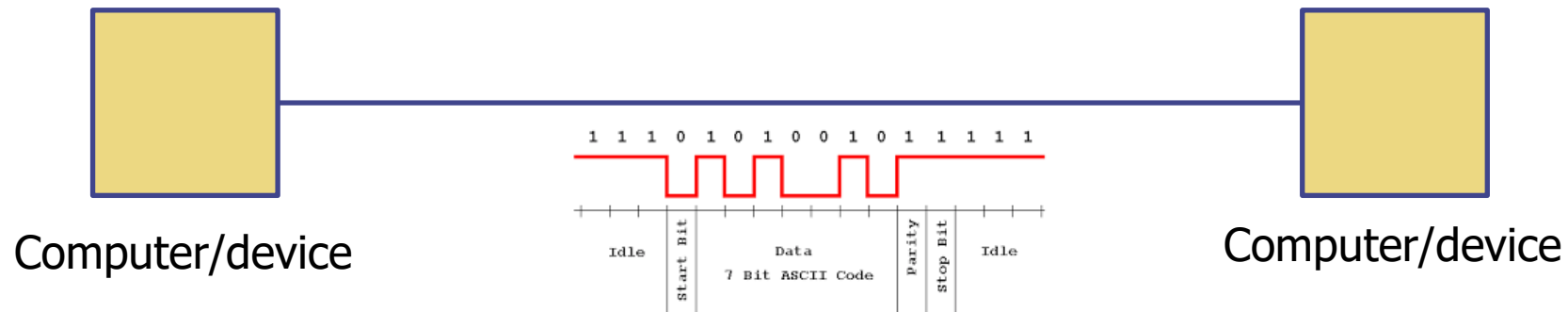
- packets will queue, waiting to be transmitted on output link
- packets can be dropped (lost) if memory (buffer) in router fills up

Subjects of today:

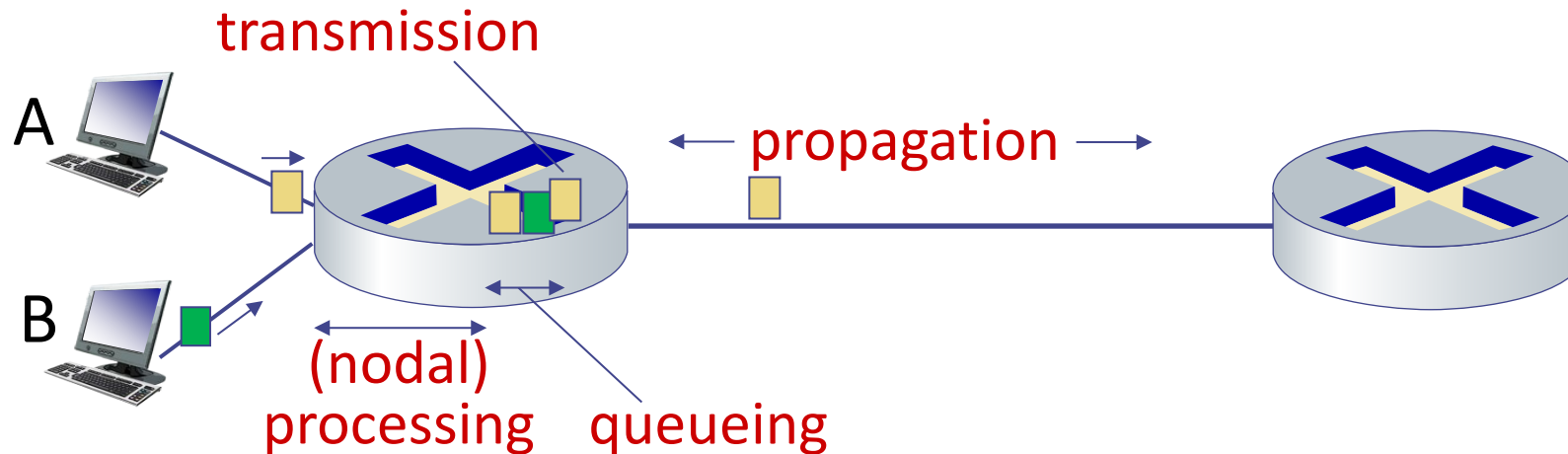
- The Internet
- The Network edge
- The Network core
- **Performance**
- The Protocol layers of the Internet
- Lab exercise

Performance

- Bandwidth
- Attenuation and noise
- Electrical limitations of receiver/transmitter (rise and fall times)
- Encoding and modulation



Packet delay: four sources



$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

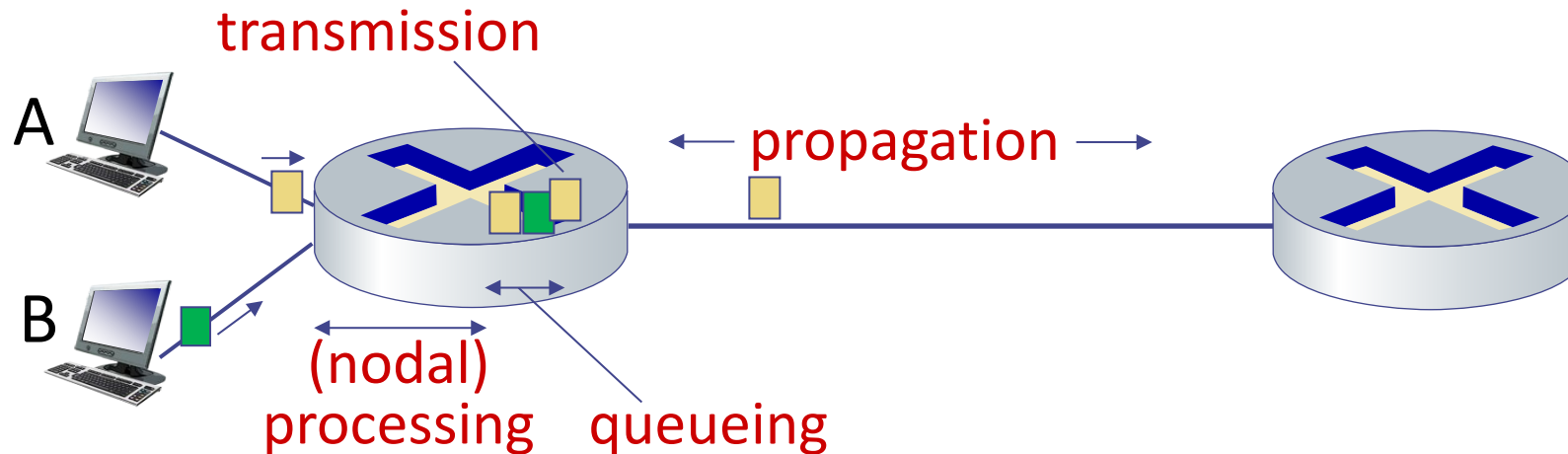
d_{proc} : processing delay

- check bit errors
- determine output link
- typically, < microsecs

d_{queue} : queueing delay

- time waiting at output link for transmission
- depends on congestion level of router

Packet delay: four sources



$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

d_{trans} : transmission delay:

- L : packet length (bits)
- R : link transmission rate (bps)

$$d_{\text{trans}} = L/R$$

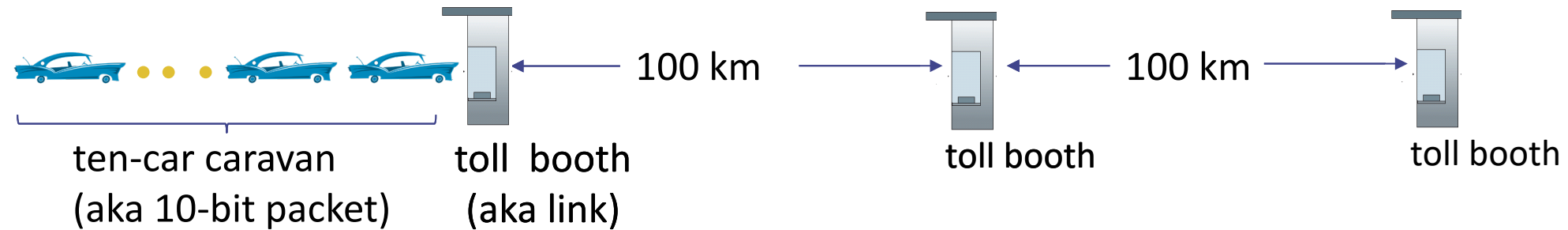
d_{prop} : propagation delay:

- d : length of physical link
- s : propagation speed ($\sim 2 \times 10^8$ m/sec)

$$d_{\text{prop}} = d/s$$

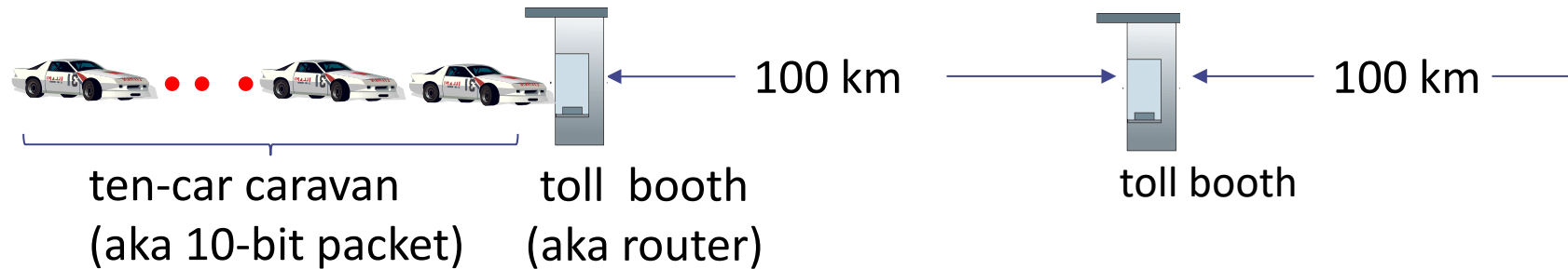
$$d_{\text{trans}} \neq d_{\text{prop}}$$

Caravan analogy

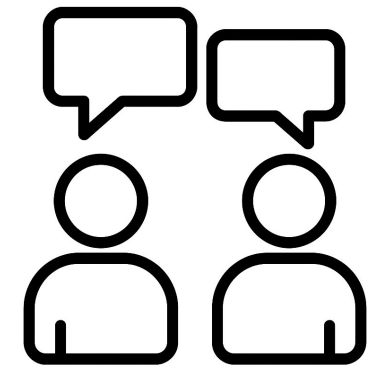


- car $\sim$ bit; caravan $\sim$ packet; toll service $\sim$ link transmission
- toll booth takes 12 sec to service a car (bit transmission time)
- Cars “propagate” at 100 km/hr
- **Q: How long until caravan is lined up before 2nd toll booth?**

Caravan analogy



- suppose cars now “propagate” at 1000 km/hr
- and suppose toll booth now takes 1 min to service a car
- **Q: Will cars arrive to 2nd booth before all cars serviced at first booth?**
A: Yes! after 7 min, first car arrives at second booth; three cars still at first booth



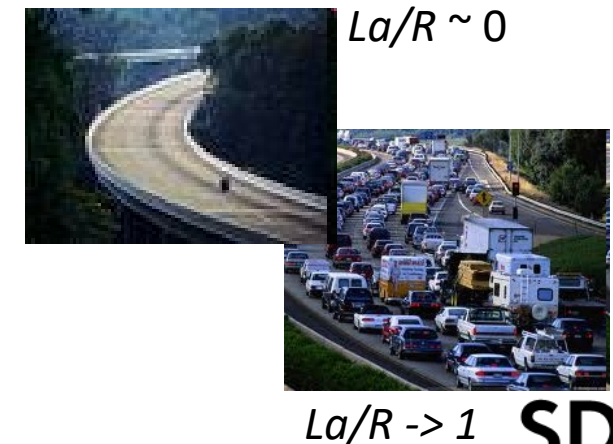
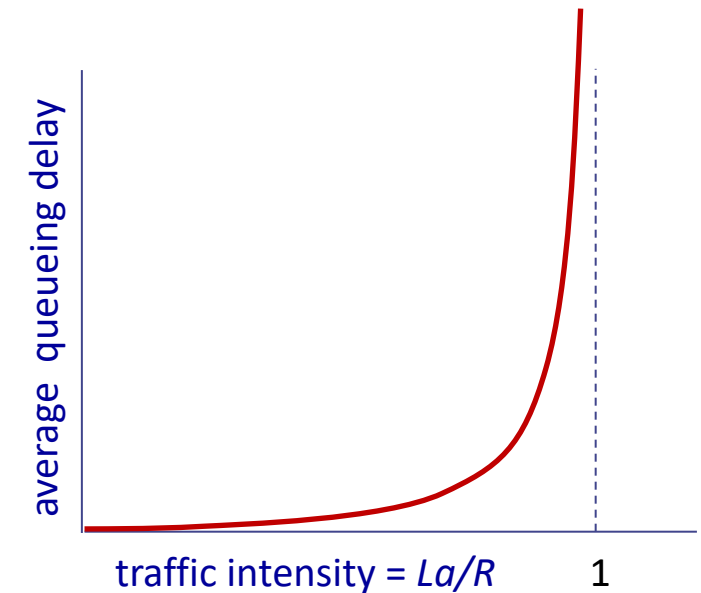
What could be analogous to processing and queuing delay?

Packet queueing delay

- a : average packet arrival rate [amount]
- L : packet length [bits]
- R : link bandwidth (bit transmission rate)

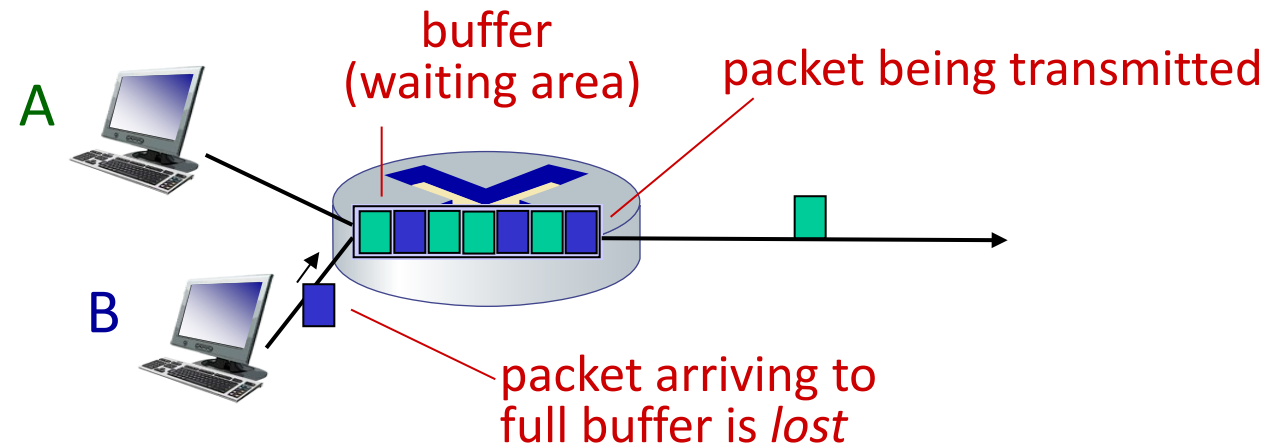
$$\frac{L \cdot a}{R} : \frac{\text{arrival rate of bits}}{\text{service rate of bits}} \quad \text{“traffic intensity”}$$

- $La/R \sim 0$: avg. queueing delay small
- $La/R \rightarrow 1$: avg. queueing delay large
- $La/R > 1$: more “work” arriving is more than can be serviced - average delay infinite!



Packet loss

- queue (aka buffer) preceding link in buffer has finite capacity
- packet arriving to full queue dropped (aka lost)
- lost packet may be retransmitted by previous node, by source end system, or not at all

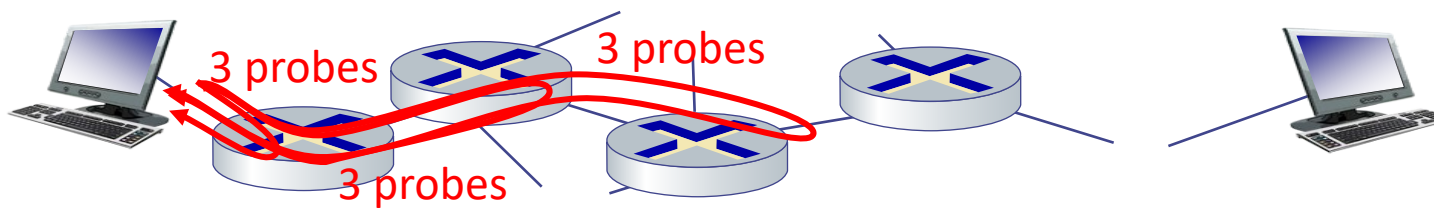


“Real” Internet delays and routes

- what do “real” Internet delay & loss look like?
- **traceroute** program: provides delay measurement from source to router along end-end Internet path towards destination.

For all i :

- sends three packets that will reach router i on path towards destination (with time-to-live field value of i)
- router i will return packets to sender
- sender measures time interval between transmission and reply



Try it out

- Open your command-line interpreter application
 - Windows: Command Prompt or Powershell
 - Run: `tracert gaia.cs.umass.edu`
 - MacOS: Terminal
 - Run: `traceroute gaia.cs.umass.edu`
 - Linux... you must know
 - `sudo apt install tracert`
 - Run: `tracert gaia.cs.umass.edu`
- *Reflect on the response times. What happens?*
- *You can also try to visualize it:*
 - <https://geotracroute.com> → Run another traceroute → Paste to “Try your own traceroute”
 - Or direct link: [Try with your own traceroute](#)

Real Internet delays and routes

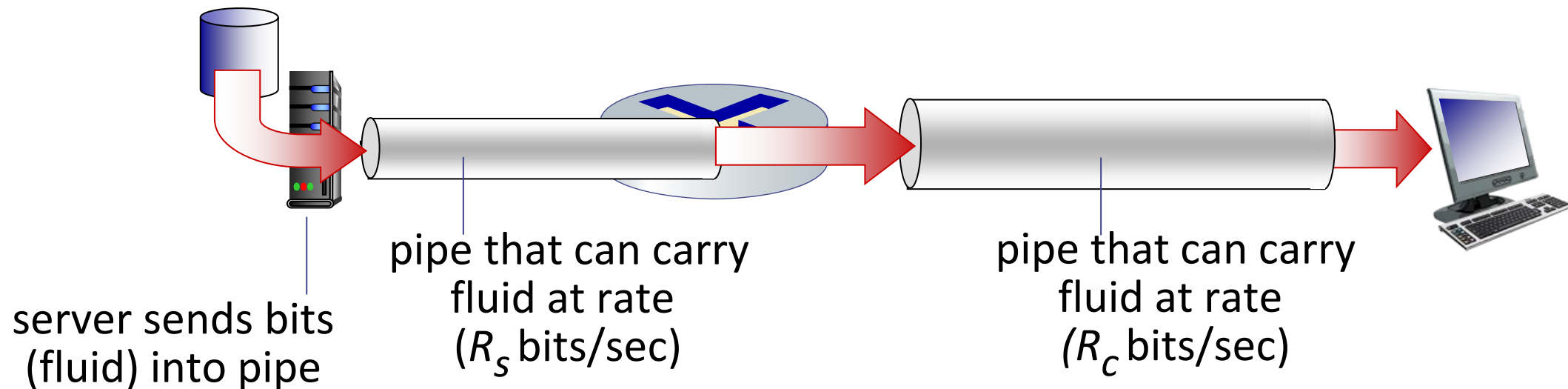
```
Trace complete.
PS C:\Users\simat> tracert gaia.cs.umass.edu

Tracing route to gaia.cs.umass.edu [128.119.245.12]
over a maximum of 30 hops:

  1    3 ms    3 ms    4 ms  10.94.64.1
  2    3 ms    3 ms    3 ms  130.226.84.233
  3    *      *      *      Request timed out.
  4    6 ms    3 ms   15 ms  netrumsydedge.sdu.dk [130.225.244.230]
  5    5 ms    6 ms    5 ms  dk-ore3.nordu.net [109.105.102.160]
  6    6 ms    7 ms    7 ms  dk-bal3.nordu.net [109.105.97.249]
  7   12 ms   11 ms   11 ms  dk-esbj.nordu.net [109.105.97.3]
  8   17 ms   17 ms   17 ms  nl-ams.nordu.net [109.105.97.75]
  9   96 ms   96 ms   98 ms  us-man2.nordu.net [109.105.97.64]
 10  105 ms  107 ms   99 ms  xe-2-3-0.118.rtr.newy32aoa.net.internet2.edu [109.105.98.10]
 11  103 ms  103 ms  105 ms  fourhundredge-0-0-0-1.4079.core1.hart2.net.internet2.edu [163.253.1.228]
 12  103 ms  106 ms  102 ms  fourhundredge-0-0-0-2.4079.core1.bost2.net.internet2.edu [163.253.2.168]
 13  108 ms  108 ms  102 ms  69.16.3.250
 14  105 ms  104 ms  104 ms  69.16.3.0
 15  105 ms  105 ms  106 ms  core2-rt-et-4-3-0.gw.umass.edu [192.80.83.105]
 16  105 ms  106 ms  106 ms  n1-rt-1-1-et-10-0-0.gw.umass.edu [128.119.0.120]
 17  104 ms  105 ms  105 ms  n1-fnt-fw-1-1-1-31-vl1092.gw.umass.edu [128.119.77.233]
 18    *      *      *      Request timed out.
 19  104 ms  111 ms  105 ms  core2-rt-et-7-2-1.gw.umass.edu [128.119.0.121]
 20  107 ms  105 ms  112 ms  n5-rt-1-1-xe-2-1-0.gw.umass.edu [128.119.3.33]
 21  105 ms  105 ms  105 ms  cics-rt-xe-0-0-0.gw.umass.edu [128.119.3.32]
 22    *      *      *      Request timed out.
 23  106 ms  104 ms  105 ms  gaia.cs.umass.edu [128.119.245.12]
```

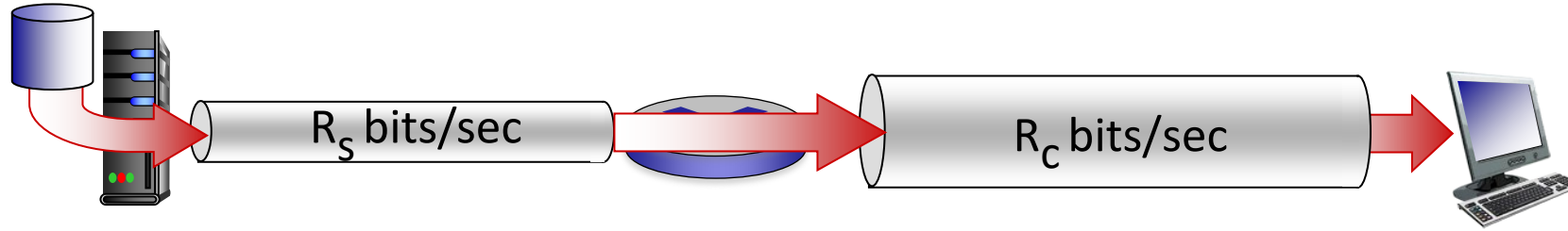
Throughput

- *throughput*: rate (bits/time unit) at which bits are being sent from sender to receiver
 - *instantaneous*: rate at given point in time
 - *average*: rate over longer period of time

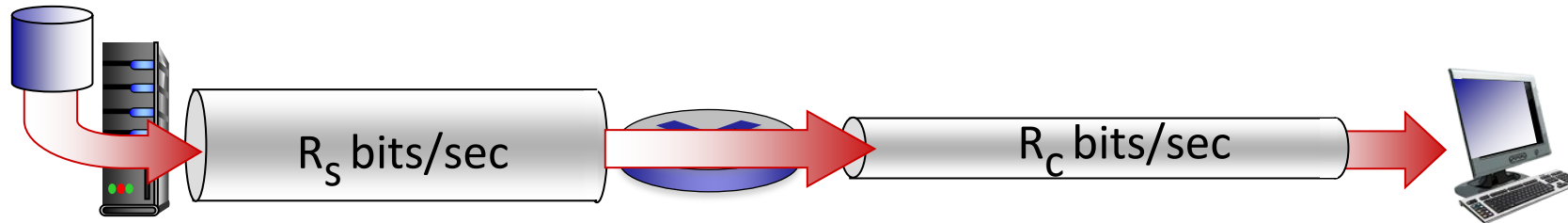


Throughput

$R_s < R_c$ What is average end-end throughput?



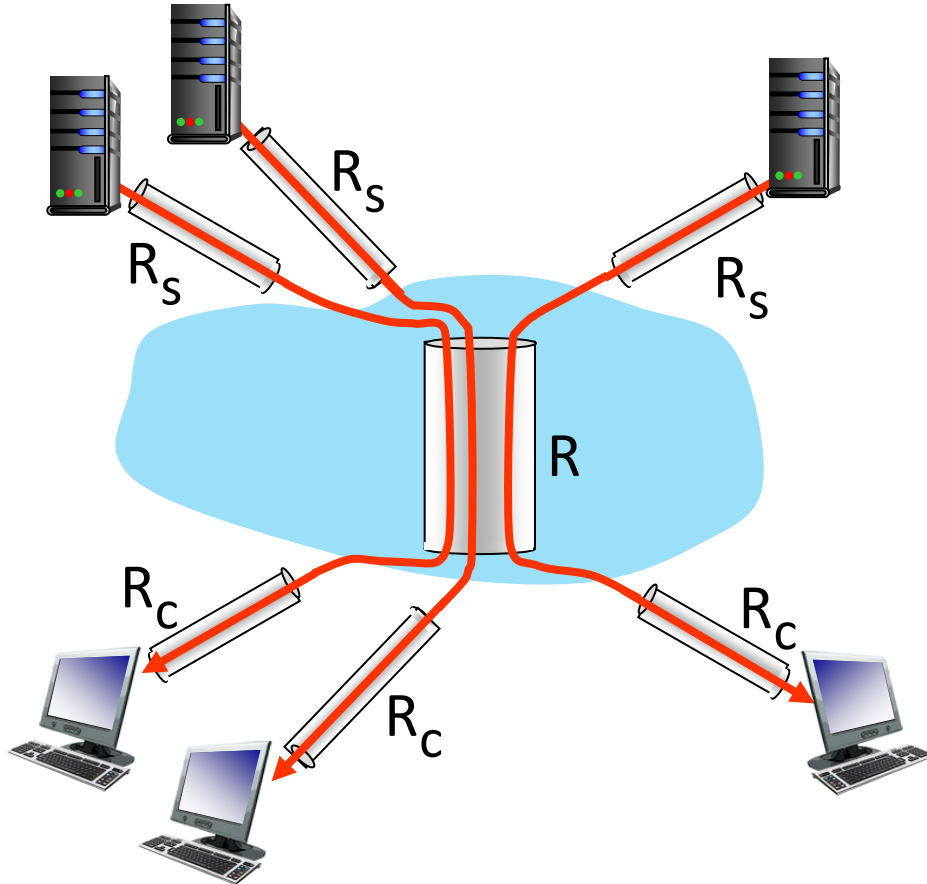
$R_s > R_c$ What is average end-end throughput?



bottleneck link

link on end-end path that constrains end-end throughput

Throughput: network scenario



- per-connection end-end throughput:
 $\min(R_c, R_s, R/10)$
- in practice: R_c or R_s is often bottleneck

* Check out the online interactive exercises for more examples:

https://gaia.cs.umass.edu/kurose_ross/interactive/

→ End-to-End Throughput (Note: round off your decimals)

Subjects of today:

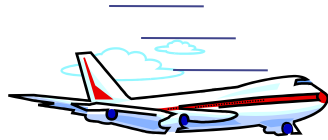
- The Internet
- The Network edge
- The Network core
- Performance
- **The Protocol layers of the Internet**
- Lab exercise

Protocol “layers” and reference models

Networks are complex,
with many “pieces”:

- hosts
- routers
- links of various media
- applications
- protocols
- hardware, software

Example: organization of air travel



————— *end-to-end transfer of person plus baggage* —————→

ticket (purchase)

baggage (check)

gates (load)

runway takeoff

airplane routing

ticket (complain)

baggage (claim)

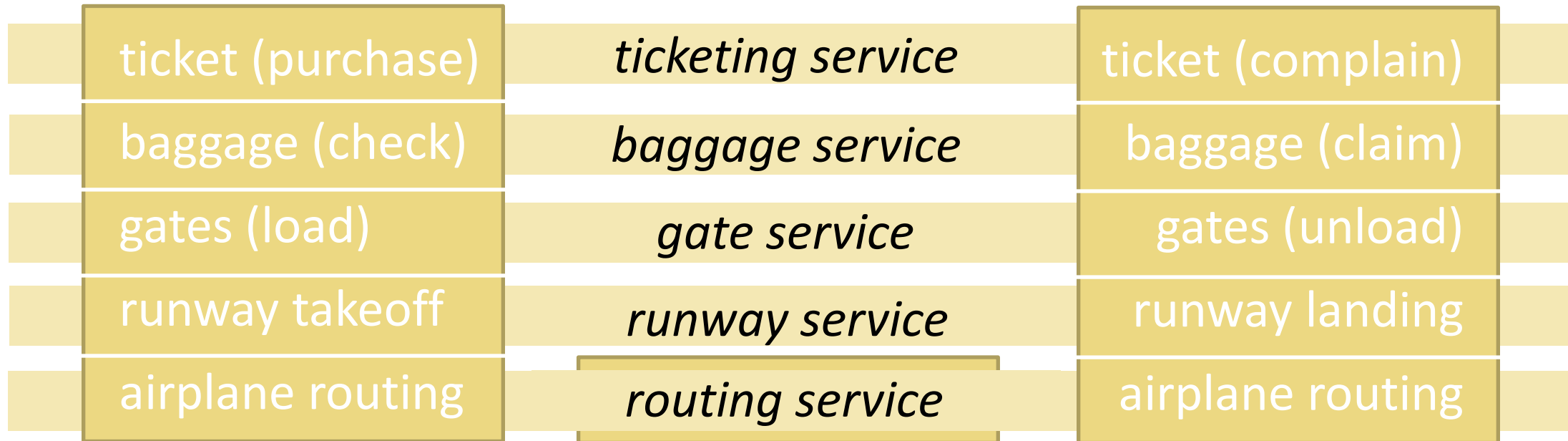
gates (unload)

runway landing

airplane routing

airplane routing

Example: organization of air travel



layers: each layer implements a service

- via its own internal-layer actions
- relying on services provided by layer below

Why layering?

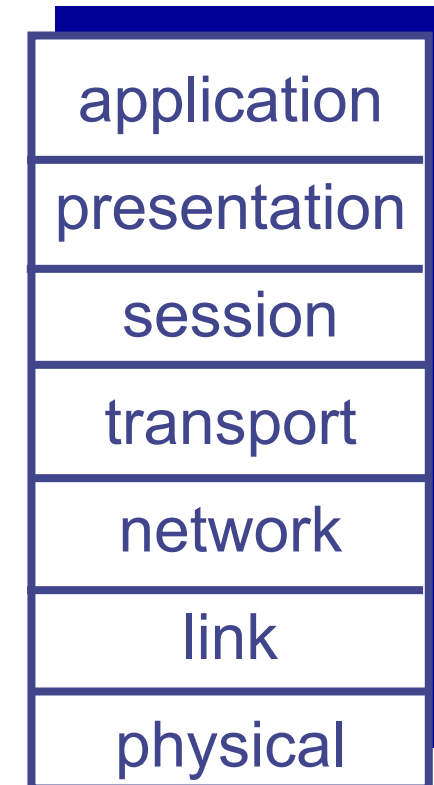
Approach to designing/discussing complex systems:

- explicit structure allows identification, relationship of system's pieces
- modularization eases maintenance, updating of system
 - change in layer's service *implementation*: transparent to rest of system
 - e.g., change in gate procedure doesn't affect rest of system

ISO/OSI reference model (revisited)

Two layers not found in the Internet protocol stack!

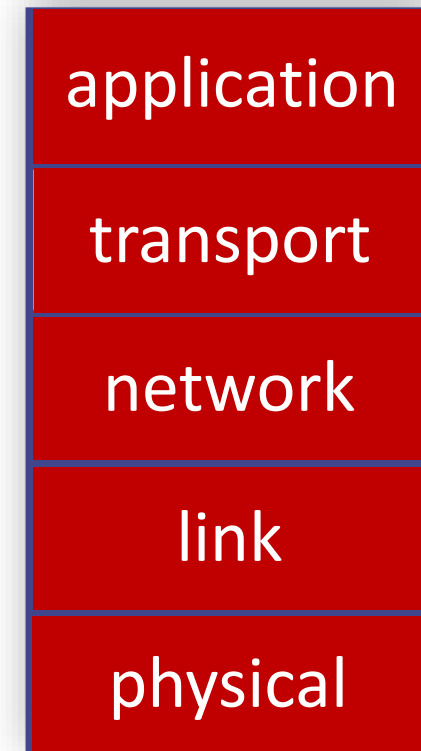
- *presentation*: allow applications to interpret meaning of data, e.g., encryption, compression, machine-specific conventions
- *session*: synchronization, checkpointing, recovery of data exchange
- Internet stack “missing” these layers!
 - these services, *if needed*, must be implemented in application



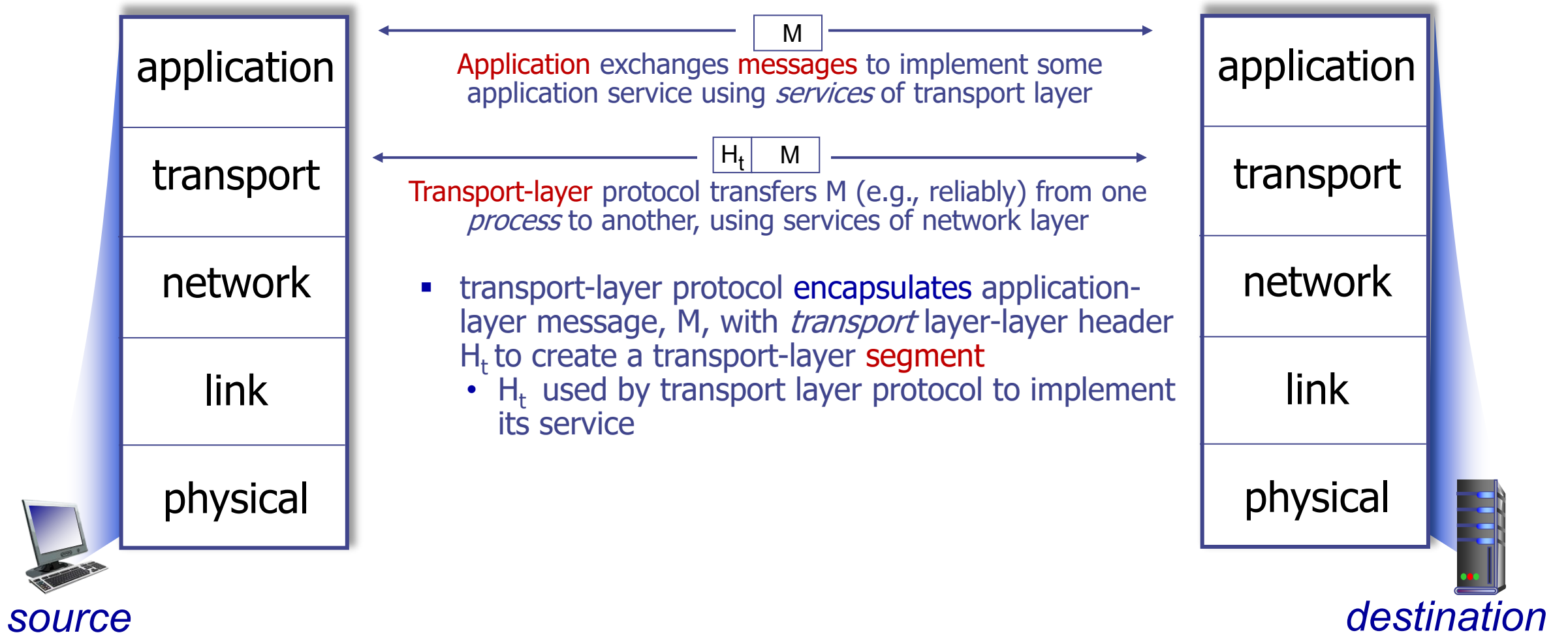
The seven layer OSI/ISO
reference model

Layered Internet protocol stack

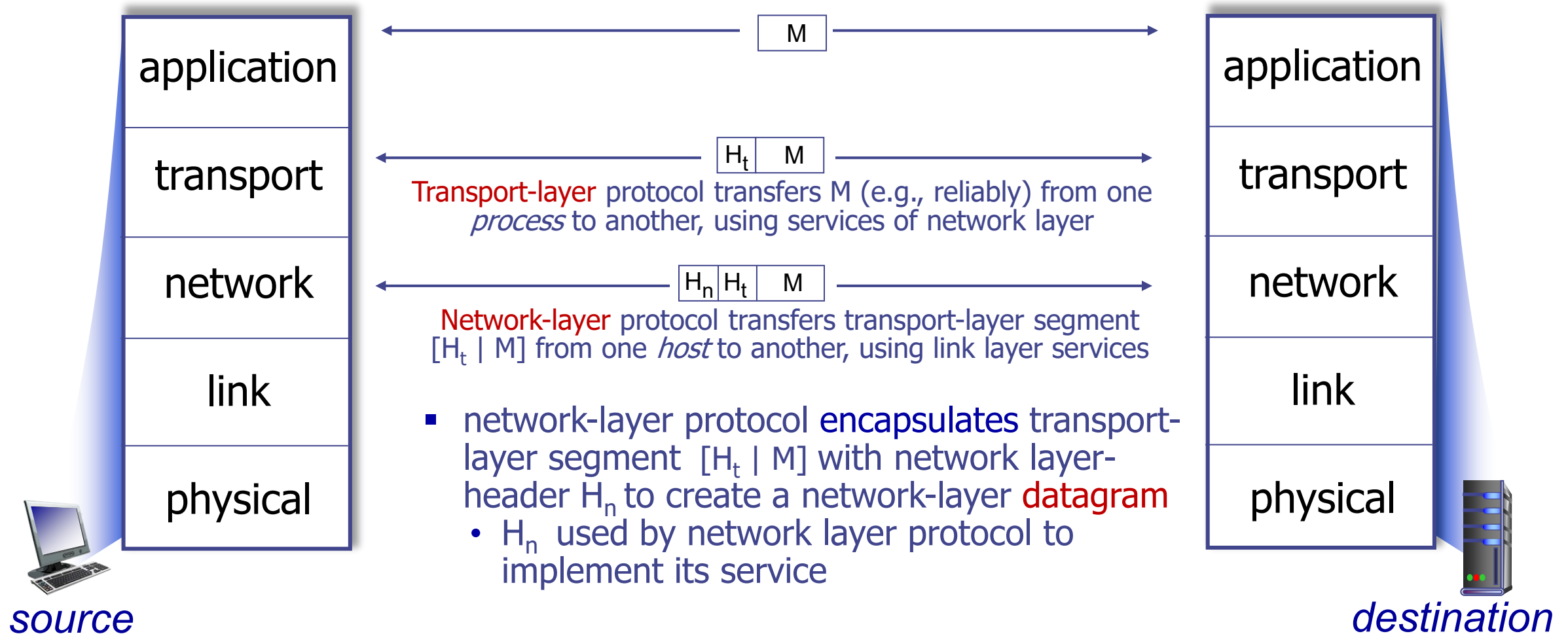
- *application*: supporting network applications
 - HTTP, IMAP, SMTP, DNS
- *transport*: process-process data transfer
 - TCP, UDP
- *network*: routing of datagrams from source to destination
 - IP, routing protocols
- *link*: data transfer between neighboring network elements
 - Ethernet, 802.11 (WiFi), PPP
- *physical*: bits “on the wire”



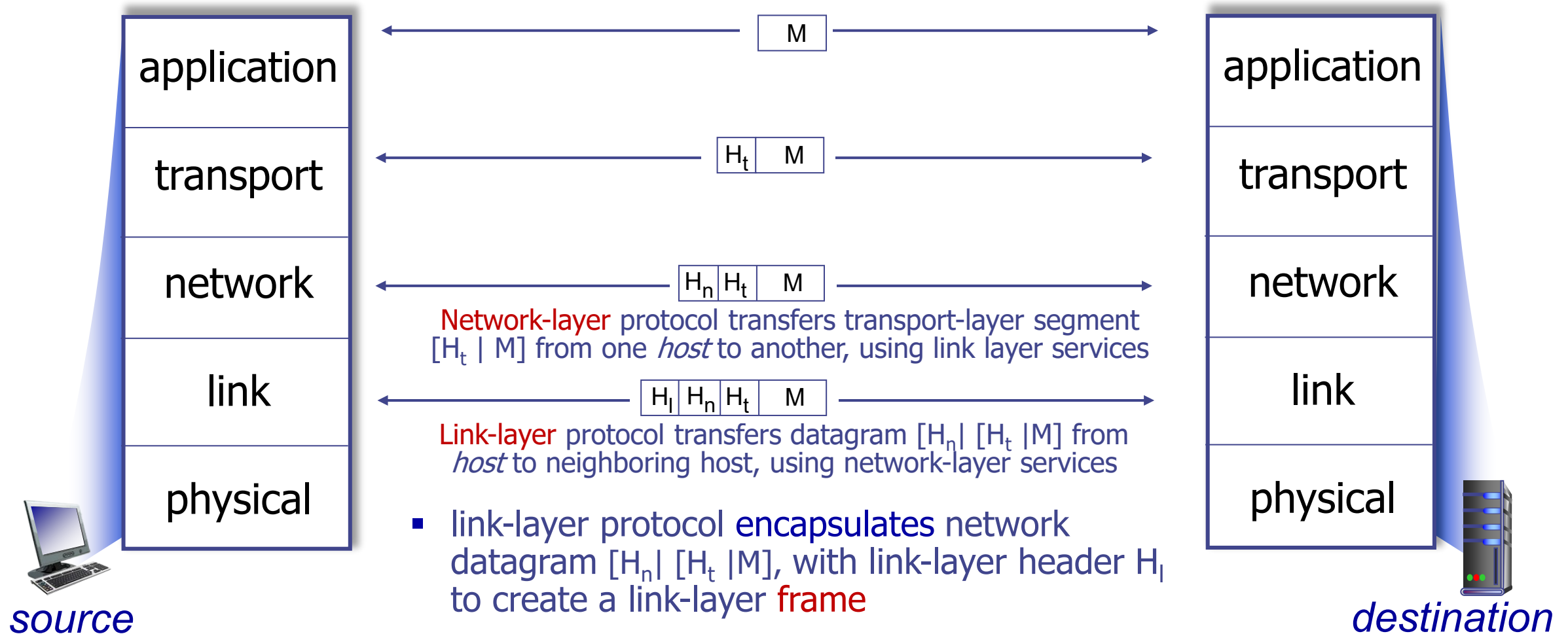
Services, Layering and Encapsulation



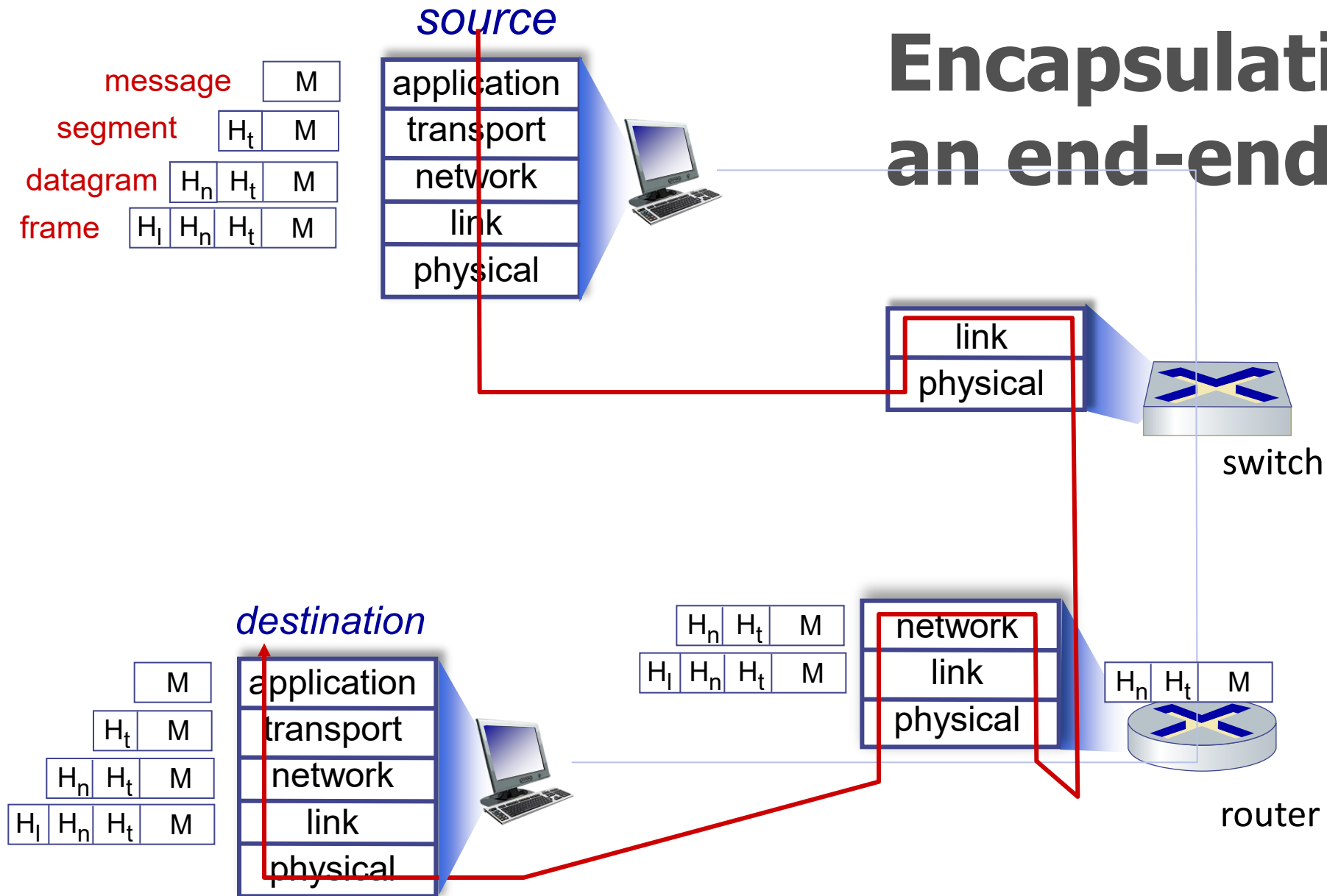
Services, Layering and Encapsulation



Services, Layering and Encapsulation



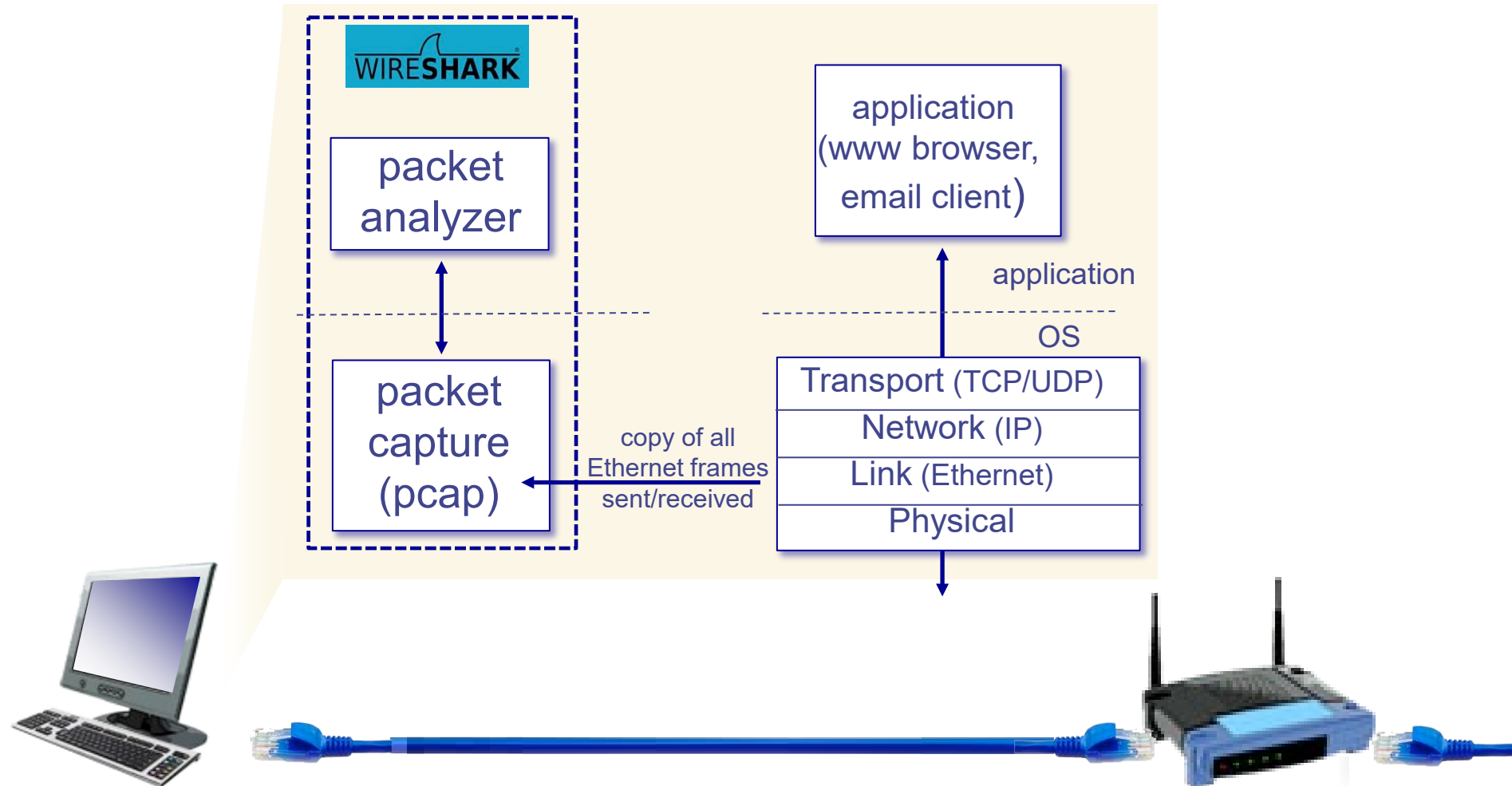
Encapsulation: an end-end view



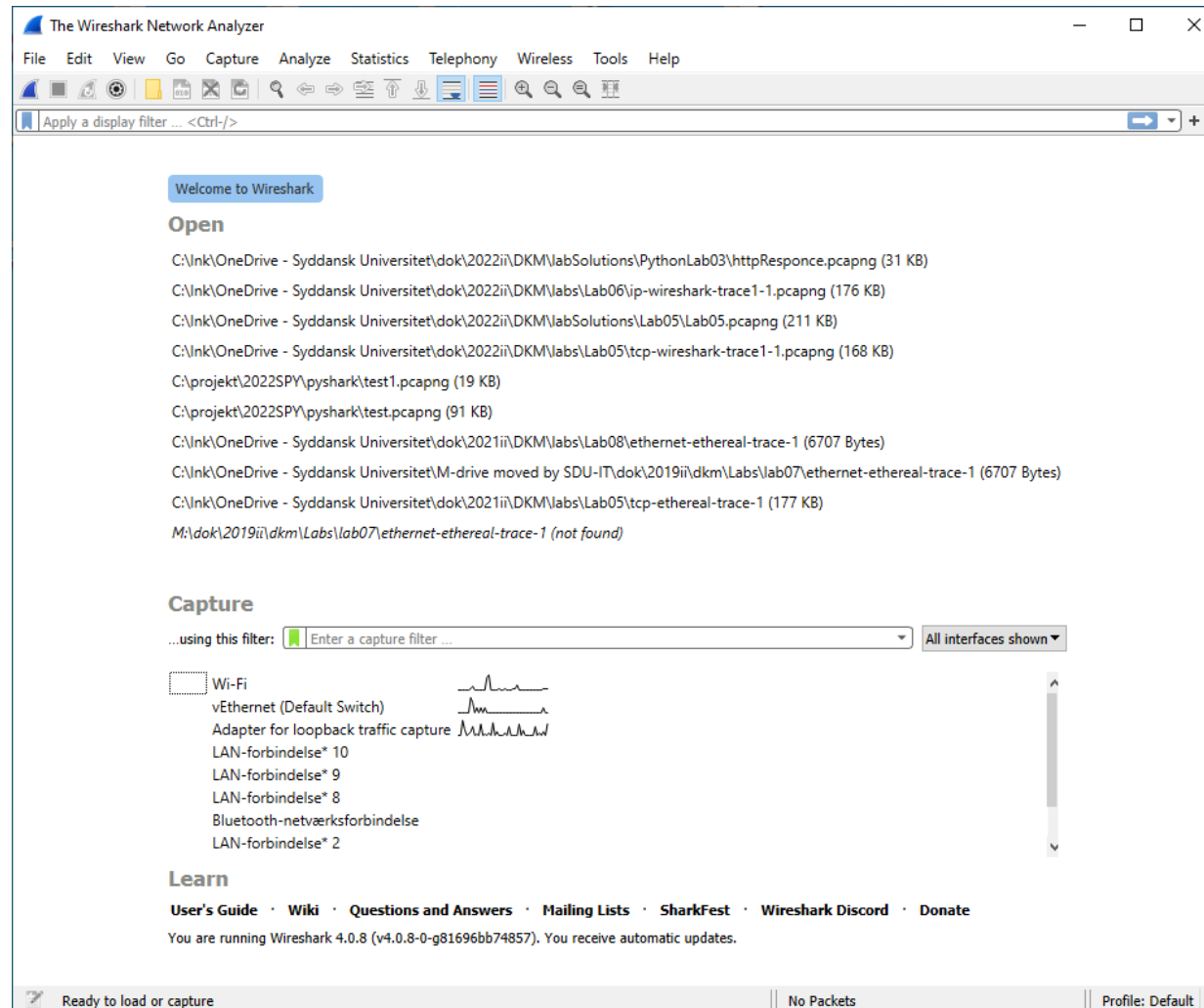
Subjects of today:

- The Internet
- The Network edge
- The Network core
- Performance
- The Protocol layers of the Internet
- **Lab exercise**

Wireshark



Wireshark



Wireshark

Menus

Filters

Packet list

Packet details

Packet content

The image shows the Wireshark network protocol analyzer interface. The top menu bar includes File, Edit, View, Go, Capture, Analyze, Statistics, Telephony, Wireless, Tools, and Help. Below the menu is a toolbar with icons for various functions. A filter bar shows 'Apply a display filter ... <Ctrl-/>'. The main packet list table displays the following data:

No.	Time	Source	Destination	Protocol	Length	Info
32	10:56:55,025847	13.107.138.8	10.94.58.75	TCP	54	443 → 52206 [ACK] Seq=1 Ack=2116
33	10:56:55,061600	52.98.149.130	10.94.58.75	TLSv1.2	97	Application Data
34	10:56:55,063475	52.98.149.130	10.94.58.75	TLSv1.2	97	Application Data
35	10:56:55,071157	13.107.138.8	10.94.58.75	TCP	1304	443 → 52206 [ACK] Seq=1 Ack=2116
36	10:56:55,071157	13.107.138.8	10.94.58.75	TLSv1.2	745	Application Data
37	10:56:55,071281	10.94.58.75	13.107.138.8	TCP	54	52206 → 443 [ACK] Seq=2116 Ack=194
38	10:56:55,112620	10.94.58.75	52.98.149.130	TCP	54	52379 → 443 [ACK] Seq=1 Ack=44 Wi
39	10:56:55,112624	10.94.58.75	52.98.149.130	TCP	54	52378 → 443 [ACK] Seq=1 Ack=44 Wi
40	10:56:55,429176	3.120.122.184	10.94.58.75	TLSv1.2	87	Application Data
41	10:56:55,482968	10.94.58.75	3.120.122.184	TCP	54	51231 → 443 [ACK] Seq=1 Ack=34 Wi
42	10:56:57,694961	10.94.58.75	10.220.2.5	DNS	88	Standard query 0x3078 TXT debug.o
43	10:56:57,848739	10.94.58.75	10.94.63.255	UDP	82	50260 → 1947 Len=40
44	10:56:57,946379	10.94.58.75	192.168.1.58	SNMP	120	get-request 1.3.6.1.2.1.25.3.2.1.!
45	10:56:57,946736	10.94.58.75	192.168.8.103	SNMP	120	get-request 1.3.6.1.2.1.25.3.2.1.!
46	10:56:59.699681	10.94.58.75	192.168.8.103	SNMP	84	get-next-request 1.3.6.1.4.1.2699

The packet details pane for packet 44 shows the following structure:

- > Frame 44: 120 bytes on wire (960 bits), 120 b
- > Ethernet II, Src: IntelCor_97:11:bb (58:a0:23:97:11:bb), Dst: 10.94.58.75
- > Internet Protocol Version 4, Src: 10.94.58.75
- > User Datagram Protocol, Src Port: 53758, Dst Port: 1947
- > Simple Network Management Protocol [Community ID: 1:eAA8yPpNoEKwNaEPNrKp2mLuD0c=]

The packet content pane shows the raw data in hexadecimal and ASCII:

```
0000 00 00 0c 07 ac 01 58 a0 23 97 11 bb 08 00 45 00  ....X.#...E.
0010 00 6a 75 38 00 00 80 11 be bf 0a 5e 3a 4b c0 a8  ..ju8.....^:K.
0020 01 3a d1 fe 00 a1 00 56 29 95 30 4c 02 01 00 04  .:.....V).0L...
0030 06 70 75 62 6c 69 63 a0 3f 02 02 08 10 02 01 00  .public?.....
0040 02 01 00 30 33 30 0f 06 0b 2b 06 01 02 01 19 03  ..030...+.....
0050 02 01 05 01 05 00 30 0f 06 0b 2b 06 01 02 01 19  .....0...+.....
0060 03 05 01 01 01 05 00 30 0f 06 0b 2b 06 01 02 01  .....0...+.....
0070 19 03 05 01 02 01 05 00  .....
```

Wireshark

- **Today: Lab 1 - Wireshark Intro**
- **Purpose:** The labs will **demonstrate the theory** we discuss in class. Doing the lab should **enhance your learning** and make it clear whether you understood the theory!
- There are no demands for hand-ins of the Labs (unless otherwise stated), but **make journals** documenting your work to the extent you find it relevant. They are useful for discussions at the class and with your classmates.
- The journal should consist of printouts and/or screenshots from Wireshark and other tools used in the lab.
- Answer the questions from the lab with references (e.g. using highlights or arrows) to the printouts/screenshots.